

✓ Лабораторная

Сидельникова Дарья

Кейс 1 и 7

Импорт библиотек

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score
```

```
np.set_printoptions(precision=6, suppress=True)
pd.set_option("display.max_columns", 120)
pd.set_option("display.width", 180)
```

Фиксируем случайность

```
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

✓ Кейс 1: интегральный оператор

Рассматривается оператор

$$(Tf)(x) = \int_0^1 \min(x, t) f(t) dt.$$

Он переводит функцию $f(t)$ в новую функцию $(Tf)(x)$. Основная задача — изучить спектральное уравнение

$$\int_0^1 \min(x, t) \varphi(t) dt = \lambda \varphi(x).$$

Здесь λ — собственное значение, а $\varphi(x)$ — собственная функция. Если функция после применения оператора меняется только в масштабе, то она является собственной функцией оператора.

Для ядра $K(x, t) = \min(x, t)$ известен аналитический ответ:

$$\lambda_k = \frac{1}{\pi^2(k - 1/2)^2},$$

$$\varphi_k(x) = \sqrt{2} \sin((k - 1/2)\pi x), \quad k = 1, 2, \dots$$

Эти формулы используются как эталон для проверки численного метода.

Симметричность и положительность ядра

Ядро симметрично, потому что $\min(x, t) = \min(t, x)$.

Ядро неотрицательно определено, так как это ковариационная функция винеровского процесса:

$$E[W(x)W(t)] = \min(x, t).$$

Поэтому для матрицы Грама $G_{ij} = \min(x_i, x_j)$ должно выполняться:

$$z^T G z \geq 0.$$

Функция ядра

ядро

$$K(x, t) = \min(x, t), \quad x, t \in [0, 1].$$

Интегральный оператор имеет вид

$$(Tf)(x) = \int_0^1 K(x, t) f(t) dt.$$

В коде ядро строится не для одной пары точек, а сразу для двух сеток.
Поэтому получается матрица

$$K_{ij} = K(x_i, t_j) = \min(x_i, t_j).$$

```
def brownian_kernel(x, t):  
    return np.minimum.outer(x, t)
```

Равномерная сетка

Для численного интегрирования непрерывный отрезок $[0, 1]$ заменяется конечным набором узлов:

$$x_i = \frac{i}{n-1}, \quad i = 0, 1, \dots, n-1.$$

Чем больше n , тем мельче шаг сетки

$$h = \frac{1}{n-1},$$

и тем точнее дискретный оператор приближает исходный интегральный оператор.

```
def make_uniform_grid(n):  
    return np.linspace(0.0, 1.0, n)
```

Матрица Грама

Матрица Грама для ядра строится по формуле

$$G_{ij} = K(x_i, x_j) = \min(x_i, x_j).$$

Если ядро симметрично и неотрицательно определено, то такая матрица должна быть симметричной и положительно полуопределённой:

$$G = G^T, \quad z^T G z \geq 0.$$

```
def build_gram_matrix(x):
    return brownian_kernel(x, x)
```

Ошибка симметричности

Симметричность матрицы проверяется через максимальное отклонение элементов от транспонированной матрицы:

$$\max_{i,j} |G_{ij} - G_{ji}|.$$

Если результат равен нулю или очень близок к нулю, матрица численно симметрична.

```
def symmetry_error(G):
    return np.max(np.abs(G - G.T))
```

Собственные значения симметричной матрицы

Для симметричной матрицы собственные значения вещественны. Если матрица положительно полуопределённая, то

$$\mu_k \geq 0.$$

Небольшие отрицательные значения порядка 10^{-12} или меньше обычно трактуются как вычислительная погрешность.

```
def symmetric_eigenvalues(G):
    return np.linalg.eigvalsh(G)
```

Проверка квадратичной формы

Неотрицательная определённость матрицы проверяется условием

$$z^T G z \geq 0$$

для любого вектора z . Это не является строгим доказательством, но является хорошей численной проверкой.

```
def random_quadratic_forms(G, n_trials=1000, seed=42):
    local_rng = np.random.default_rng(seed)
    values = []
    for _ in range(n_trials):
        z = local_rng.normal(size=G.shape[0])
        values.append(z @ G @ z)
    return np.array(values)
```

Численная проверка матрицы Грама

В этом блоке строится сетка, затем по ней строится матрица

$G_{ij} = \min(x_i, x_j)$. После этого проверяются два свойства: симметричность и неотрицательность собственных значений.

```
x = make_uniform_grid(50)
G = build_gram_matrix(x)

print("Размер G:", G.shape)
print("Ошибка симметричности:", symmetry_error(G))

vals = symmetric_eigenvalues(G)
print("Минимальное собственное значение:", vals.min())
print("10 наибольших собственных значений:")
print(vals[::-1][:10])
```

```
Размер G: (50, 50)
Ошибка симметричности: 0.0
Минимальное собственное значение: 0.0
10 наибольших собственных значений:
[20.268005  2.253513  0.812355  0.415303  0.251909  0.169201  0.1216
 0.091793  0.071851  0.057871]
```

Матрица G имеет правильный размер и численно является симметричной. Минимальное собственное значение не уходит существенно ниже нуля, поэтому численная проверка согласуется с теорией: ядро $K(x, t) = \min(x, t)$ задаёт положительно полуопределённую матрицу Грама.

Проверка `z.T @ G @ z >= 0`

Численная проверка положительности через случайные векторы

Здесь проверяется величина

$$z^T G z.$$

Если матрица действительно положительно полуопределённая, то для всех случайно выбранных z эта величина должна быть неотрицательной.

```
q = random_quadratic_forms(G, n_trials=1000, seed=RANDOM_STATE)

print("Минимальное z^T G z:", q.min())
print("Количество отрицательных значений меньше -1e-10:", np.sum(q
```

```
Минимальное z^T G z: 1.1950920429433005
Количество отрицательных значений меньше -1e-10: 0
```

Для случайных векторов значения $z^T G z$ не становятся отрицательными с существенной погрешностью

Переход к дифференциальной задаче

Раскладываем интеграл:

$$\int_0^1 \min(x, t) \varphi(t) dt = \int_0^x t \varphi(t) dt + x \int_x^1 \varphi(t) dt.$$

Обозначим левую часть через $F(x)$. Тогда:

$$F(x) = \lambda \varphi(x).$$

$$F'(x) = \int_x^1 \varphi(t) dt.$$

$$F''(x) = -\varphi(x).$$

Так как $F(x) = \lambda \varphi(x)$, получаем:

$$\lambda \varphi''(x) + \varphi(x) = 0.$$

Краевые условия:

$$\varphi(0) = 0, \quad \varphi'(1) = 0.$$

Аналитическая формула собственных значений

После перехода от интегральной задачи к краевой задаче получается уравнение

$$\lambda \varphi''(x) + \varphi(x) = 0, \\ \varphi(0) = 0, \quad \varphi'(1) = 0.$$

Его собственные значения равны

$$\lambda_k = \frac{1}{\pi^2(k - 1/2)^2}, \quad k = 1, 2, \dots$$

```
def exact_eigenvalues(k_max):
    k = np.arange(1, k_max + 1)
    return 1.0 / (np.pi**2 * (k - 0.5)**2)
```

Собственные функции имеют вид

$$\varphi_k(x) = \sqrt{2} \sin \left((k - 1/2)\pi x \right).$$

Множитель $\sqrt{2}$ нужен для нормировки:

$$\int_0^1 \varphi_k^2(x) dx = 1.$$

```
def exact_eigenfunction(x, k):
    return np.sqrt(2.0) * np.sin((k - 0.5) * np.pi * x)
```

В пространстве $L^2[0, 1]$ скалярное произведение задаётся формулой

$$\langle f, g \rangle = \int_0^1 f(x)g(x) dx.$$

Численно интеграл заменяется квадратурной формулой. Здесь используется метод трапеций.

```
def numerical_inner_product(f, g, x):
    return np.trapz(f * g, x)
```

Проверка ортонормированности

Для ортонормированной системы должно выполняться

$$\langle \varphi_i, \varphi_j \rangle = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Поэтому ожидаем единицы на диагонали и нули вне диагонали.


```

x_dense = make_uniform_grid(2000)
k_max = 5
inner = np.zeros((k_max, k_max))

for i in range(1, k_max + 1):
    for j in range(1, k_max + 1):
        inner[i-1, j-1] = numerical_inner_product(
            exact_eigenfunction(x_dense, i),
            exact_eigenfunction(x_dense, j),
            x_dense,
        )

pd.DataFrame(inner, index=[f"phi_{i}" for i in range(1, k_max+1)],

```

/tmp/ipykernel_1875/3171511105.py:3: DeprecationWarning: `trapz` is
return np.trapz(f * g, x)

	phi_1	phi_2	phi_3	phi_4	phi_5
phi_1	1.000000e+00	1.110223e-16	-9.714451e-17	5.551115e-17	-6.938894e-17
phi_2	1.110223e-16	1.000000e+00	1.110223e-16	-4.163336e-17	5.551115e-17
phi_3	-9.714451e-17	1.110223e-16	1.000000e+00	1.110223e-16	-2.775558e-17
phi_4	5.551115e-17	-4.163336e-17	1.110223e-16	1.000000e+00	5.551115e-17

Функции $\phi_1, \dots, \phi_5$ ортонормированы.

На диагонали стоят единицы — значит, норма каждой функции равна 1. Вне диагонали значения почти нулевые, порядка 10^{-16} , это обычная вычислительная погрешность.

Итог: численная проверка подтверждает теорию — собственные функции взаимно ортогональны и имеют единичную норму.

Интеграл заменяется суммой

$$\int_0^1 f(t) dt \approx \sum_{j=1}^n w_j f(t_j).$$

В простом методе прямоугольников веса почти одинаковые. Метод самый простой, но обычно менее точный, чем трапеции и Симпсон.

```
def rectangle_weights(n):
    # Простая формула: всем точкам одинаковый вес
    return np.ones(n) / n
```

Веса метода трапеций

Для равномерной сетки веса метода трапеций равны

$$w_0 = w_{n-1} = \frac{h}{2}, \quad w_j = h \quad (j = 1, \dots, n-2).$$

Краевые точки получают половинный вес, потому что на концах участвуют только в одной трапеции.

```
def trapezoid_weights(n):
    # На концах веса в два раза меньше
    h = 1.0 / (n - 1)
    w = np.ones(n) * h
    w[0] = h / 2
    w[-1] = h / 2
    return w
```

Веса метода Симпсона

Метод Симпсона использует параболическую аппроксимацию подынтегральной функции. Для равномерной сетки веса имеют вид

$$\frac{h}{3}(1, 4, 2, 4, 2, \dots, 4, 1).$$

Для него нужно нечётное число узлов, потому что число интервалов должно быть чётным.

```
def simpson_weights(n):
    # Для Симпсона число интервалов должно быть чётным, значит n не
    if (n - 1) % 2 != 0:
        raise ValueError("Для метода Симпсона n должно быть нечётно")
    h = 1.0 / (n - 1)
    w = np.ones(n)
    w[1:-1:2] = 4
    w[2:-1:2] = 2
    return w * h / 3
```

Выбор квадратурной формулы

Одна и та же задача дискретизации может быть решена разными квадратурными формулами. Поэтому удобно написать отдельную функцию, которая по названию метода возвращает нужные веса.

```
def get_weights(n, method):
    if method == "rectangle":
        return rectangle_weights(n)
    if method == "trapezoid":
        return trapezoid_weights(n)
    if method == "simpson":
        return simpson_weights(n)
    raise ValueError("Неизвестный метод")
```

Матрицы K , A , B

$$A_{ij} = K(x_i, x_j)w_j,$$

$$B = W^{1/2} K W^{1/2}.$$

Матрица B симметрична, поэтому спектр удобнее считать через неё.

Матрицы дискретного оператора

Интегральное уравнение

$$\int_0^1 K(x, t)\varphi(t) dt = \lambda\varphi(x)$$

после квадратуры переходит в матричную задачу

$$\sum_j K(x_i, x_j)w_j\varphi(x_j) = \lambda\varphi(x_i).$$

Отсюда появляется матрица

$$A_{ij} = K(x_i, x_j)w_j.$$

Для симметричного вычисления спектра используется подобная симметричная матрица

$$B = W^{1/2} K W^{1/2}.$$

```
def build_operator_matrices(x, w):
    K = brownian_kernel(x, x)
    A = K * w[None, :]
    sqrt_w = np.sqrt(w)
    B = sqrt_w[:, None] * K * sqrt_w[None, :]
    return K, A, B
```

Численный спектр оператора

Матрица B симметрична, поэтому её собственные значения устойчиво считаются через `np.linalg.eigh`. После вычисления значения сортируются по убыванию, потому что первые собственные значения дают основной вклад в оператор.

```
def compute_discrete_spectrum(x, w):
    K, A, B = build_operator_matrices(x, w)
    eigvals, eigvecs = np.linalg.eigh(B)

    order = np.argsort(eigvals)[::-1]
    eigvals = eigvals[order]
    eigvecs = eigvecs[:, order]

    phi = eigvecs / np.sqrt(w[:, None])
    return eigvals, phi
```

Дискретная норма L^2

Непрерывная норма

$$\|\varphi\|_{L^2} = \left(\int_0^1 \varphi^2(x) dx \right)^{1/2}$$

заменяется дискретной нормой

$$\|\varphi\|_w = \left(\sum_j \varphi^2(x_j) w_j \right)^{1/2}.$$

```
def discrete_l2_norm(phi, w):
    return np.sqrt(np.sum(phi**2 * w))
```

Нормировка численных собственных функций

После вычисления собственных векторов их нужно привести к условию

$$\sum_j \varphi_k^2(x_j) w_j \approx 1.$$

Это делает численные функции сопоставимыми с аналитическими.

```
def normalize_discrete_functions(phi, w):
    phi = phi.copy()
    for k in range(phi.shape[1]):
        norm = discrete_l2_norm(phi[:, k], w)
        if norm > 0:
            phi[:, k] /= norm
    return phi
```

Выбор ориентации знака

Собственная функция определена с точностью до знака. Если φ является собственной функцией, то $-\varphi$ тоже является собственной функцией.

```
def orient_function_signs(phi, x):
    phi = phi.copy()
    mid = np.argmin(np.abs(x - 0.5))
    for k in range(phi.shape[1]):
        if phi[mid, k] < 0:
            phi[:, k] *= -1
    return phi
```

Ошибка собственной функции

Для сравнения аналитической и численной функции используется дискретная L^2 -ошибка:

$$e_k^{(\varphi)} = \left(\sum_j |\varphi_k^{num}(x_j) - \varphi_k^{exact}(x_j)|^2 w_j \right)^{1/2}.$$

```
def eigenfunction_error(phi_num, phi_exact, w):
    return np.sqrt(np.sum((phi_num - phi_exact)**2 * w))
```

Сравнение численных и аналитических собственных значений

В этом блоке вычисляются первые собственные значения численного оператора и сравниваются с точными значениями. Абсолютная ошибка считается по формуле

$$e_k^{(\lambda)} = |\lambda_k^{num} - \lambda_k^{exact}|.$$

```
n = 101
x = make_uniform_grid(n)
w = get_weights(n, "trapezoid")

eig_num, phi_num = compute_discrete_spectrum(x, w)
phi_num = normalize_discrete_functions(phi_num, w)
phi_num = orient_function_signs(phi_num, x)

eig_exact = exact_eigenvalues(6)

pd.DataFrame({
    "k": np.arange(1, 7),
    "lambda_exact": eig_exact,
    "lambda_num": eig_num[:6],
    "abs_error": np.abs(eig_num[:6] - eig_exact),
})
```

	k	lambda_exact	lambda_num	abs_error
0	1	0.405285	0.405293	0.000008
1	2	0.045032	0.045040	0.000008
2	3	0.016211	0.016220	0.000008
3	4	0.008271	0.008279	0.000008
4	5	0.005004	0.005012	0.000008
5	6	0.003349	0.003358	0.000008



Численные собственные значения почти совпадают с аналитическими.

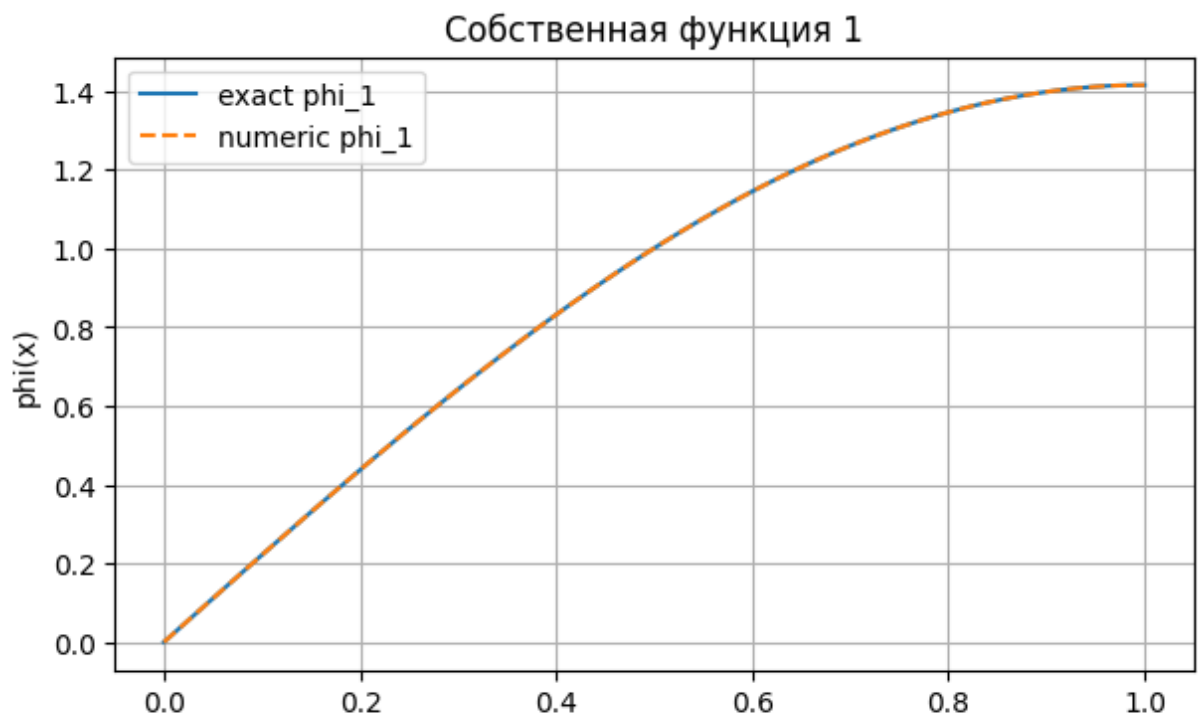
Абсолютная ошибка для всех первых 6 значений очень маленькая — около $8 \cdot 10^{-6}$. Это означает, что дискретизация оператора выполнена корректно, а численный метод хорошо приближает теоретический спектр.

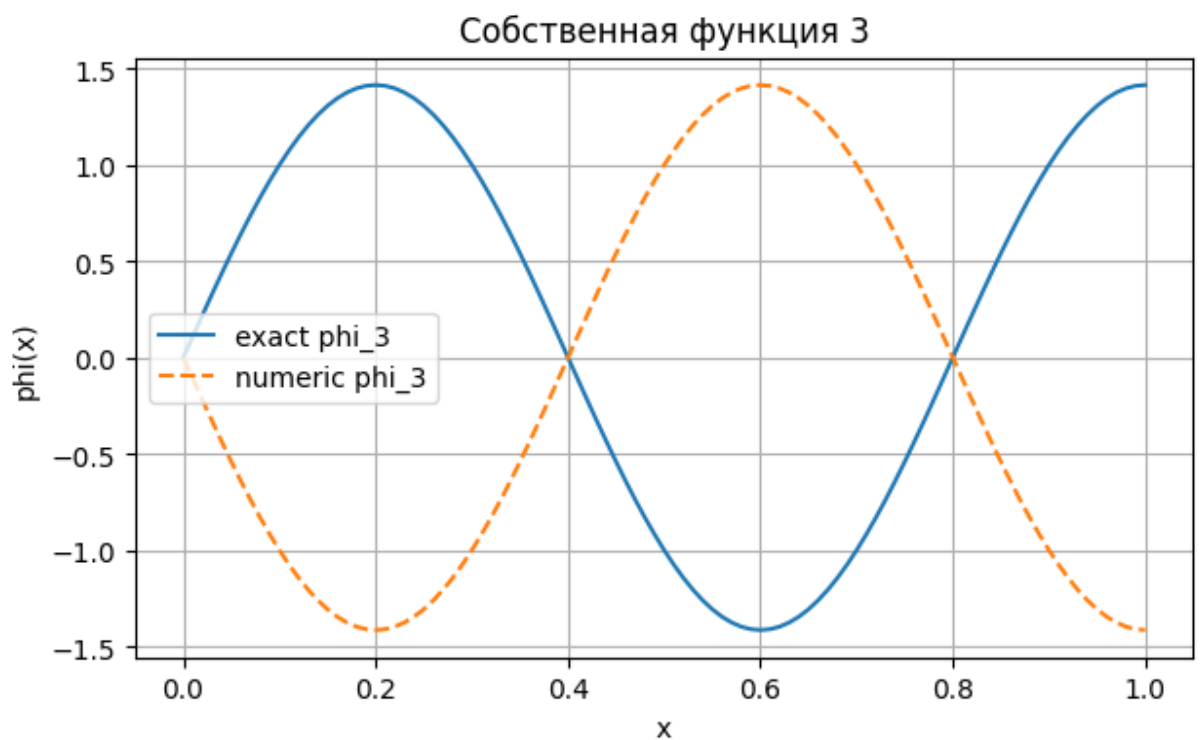
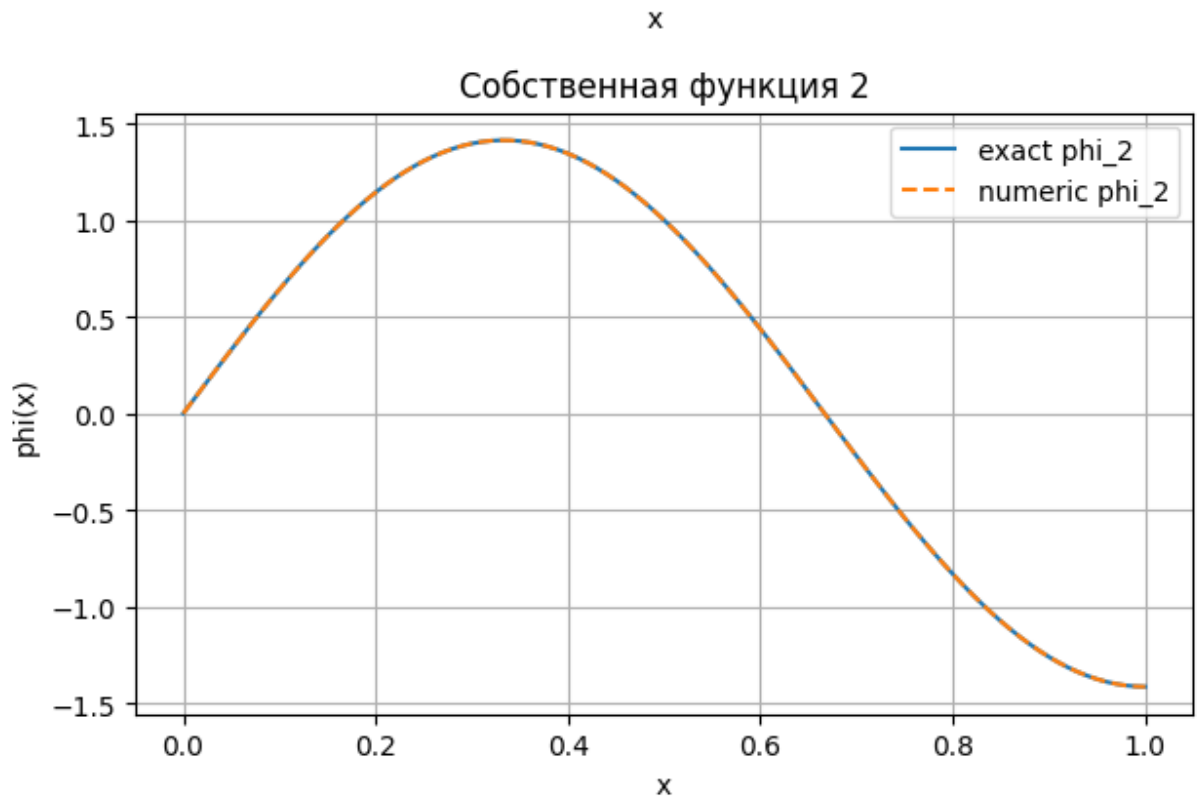
результаты подтверждают правильность построенной матрицы оператора и корректность вычисления собственных значений.

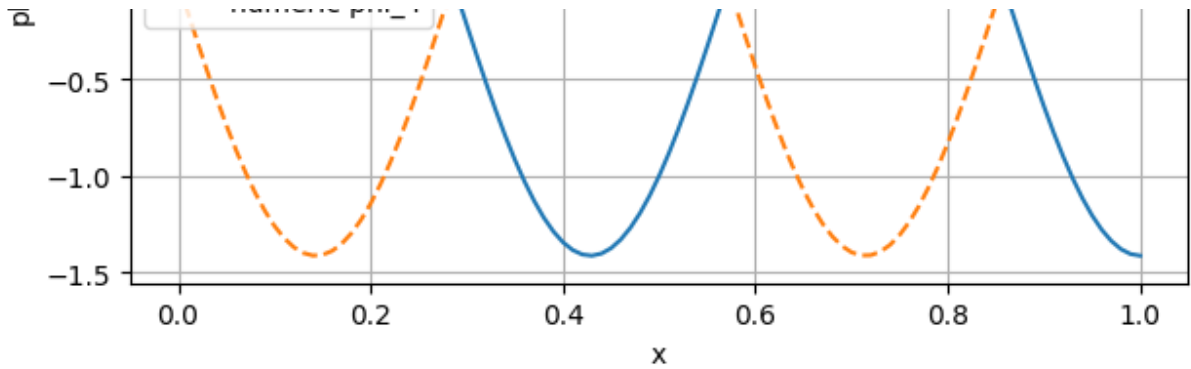
Графики первых собственных функций

На каждом графике сравниваются две кривые: аналитическая функция $\varphi_k(x)$ и численно восстановленная функция. Если линии совпадают или отличаются только знаком, численная функция восстановлена корректно.

```
for k in range(1, 5):  
    plt.figure(figsize=(7, 4))  
    plt.plot(x, exact_eigenfunction(x, k), label=f"exact phi_{k}")  
    plt.plot(x, phi_num[:, k-1], "--", label=f"numeric phi_{k}")  
    plt.title(f"Собственная функция {k}")  
    plt.xlabel("x")  
    plt.ylabel("phi(x)")  
    plt.grid(True)  
    plt.legend()  
    plt.show()
```







Численные собственные функции хорошо совпадают с аналитическими.

Для ϕ_1 и ϕ_2 совпадение почти полное: линии практически накладываются друг на друга.

Для ϕ_3 и ϕ_4 численная функция отличается знаком от аналитической. Это не ошибка, потому что собственная функция определена с точностью до знака: если $\phi(x)$ — собственная функция, то $-\phi(x)$ тоже собственная функция.

Итог: численный метод корректно восстанавливает форму первых собственных функций, а расхождение по знаку не влияет на правильность результата.

Таблица ошибок для разных размеров сетки

Здесь исследуется, как точность зависит от числа узлов n . При увеличении n шаг сетки уменьшается, поэтому ожидается уменьшение ошибки собственных значений.

```
def spectrum_error_table(n_values, method="trapezoid", k_max=5):
    rows = []
    exact = exact_eigenvalues(k_max)
    for n in n_values:
        x = make_uniform_grid(n)
        w = get_weights(n, method)
        eig_num, _ = compute_discrete_spectrum(x, w)
        for k in range(1, k_max + 1):
            rows.append({
                "method": method,
                "n": n,
                "k": k,
                "lambda_exact": exact[k-1],
                "lambda_num": eig_num[k-1],
                "abs_error": abs(eig_num[k-1] - exact[k-1]),
            })
    return pd.DataFrame(rows)
```

Запуск эксперимента по сеткам

Для каждого значения n строится свой дискретный оператор, затем вычисляются первые собственные значения и абсолютные ошибки. В этой таблице можно увидеть сходимость метода.

Рассматривала и прокручивала все, оставила в ячейке метод Трапеции

```
n_values = [21, 51, 101, 201]
err_spectrum = spectrum_error_table(n_values, method="trapezoid", k_
err_spectrum
```



	method	n	k	lambda_exact	lambda_num	abs_error
0	trapezoid	21	1	0.405285	0.405493	0.000208
1	trapezoid	21	2	0.045032	0.045241	0.000209
2	trapezoid	21	3	0.016211	0.016421	0.000210
3	trapezoid	21	4	0.008271	0.008483	0.000212
4	trapezoid	21	5	0.005004	0.005217	0.000214
5	trapezoid	51	1	0.405285	0.405318	0.000033
6	trapezoid	51	2	0.045032	0.045065	0.000033
7	trapezoid	51	3	0.016211	0.016245	0.000033
8	trapezoid	51	4	0.008271	0.008305	0.000033
9	trapezoid	51	5	0.005004	0.005037	0.000033
10	trapezoid	101	1	0.405285	0.405293	0.000008
11	trapezoid	101	2	0.045032	0.045040	0.000008
12	trapezoid	101	3	0.016211	0.016220	0.000008
13	trapezoid	101	4	0.008271	0.008279	0.000008
14	trapezoid	101	5	0.005004	0.005012	0.000008
15	trapezoid	201	1	0.405285	0.405287	0.000002
16	trapezoid	201	2	0.045032	0.045034	0.000002
17	trapezoid	201	3	0.016211	0.016213	0.000002
18	trapezoid	201	4	0.008271	0.008273	0.000002
19	trapezoid	201	5	0.005004	0.005006	0.000002

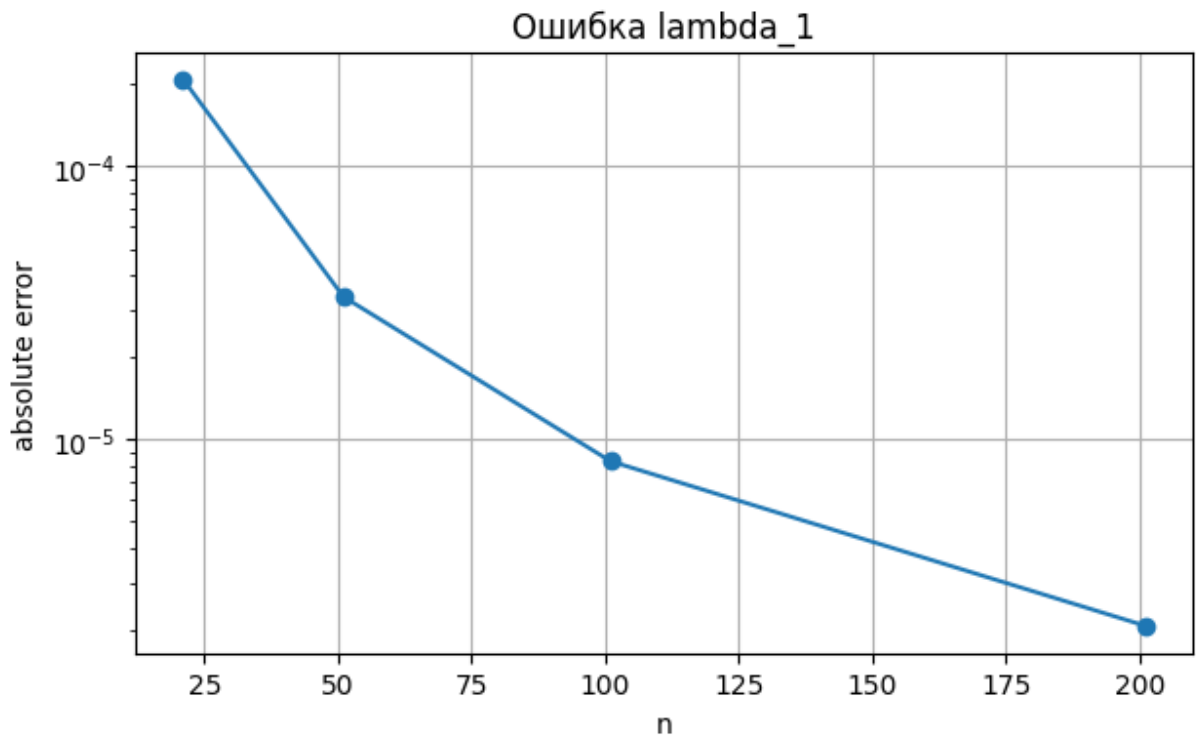
Далее: [New interactive sheet](#)

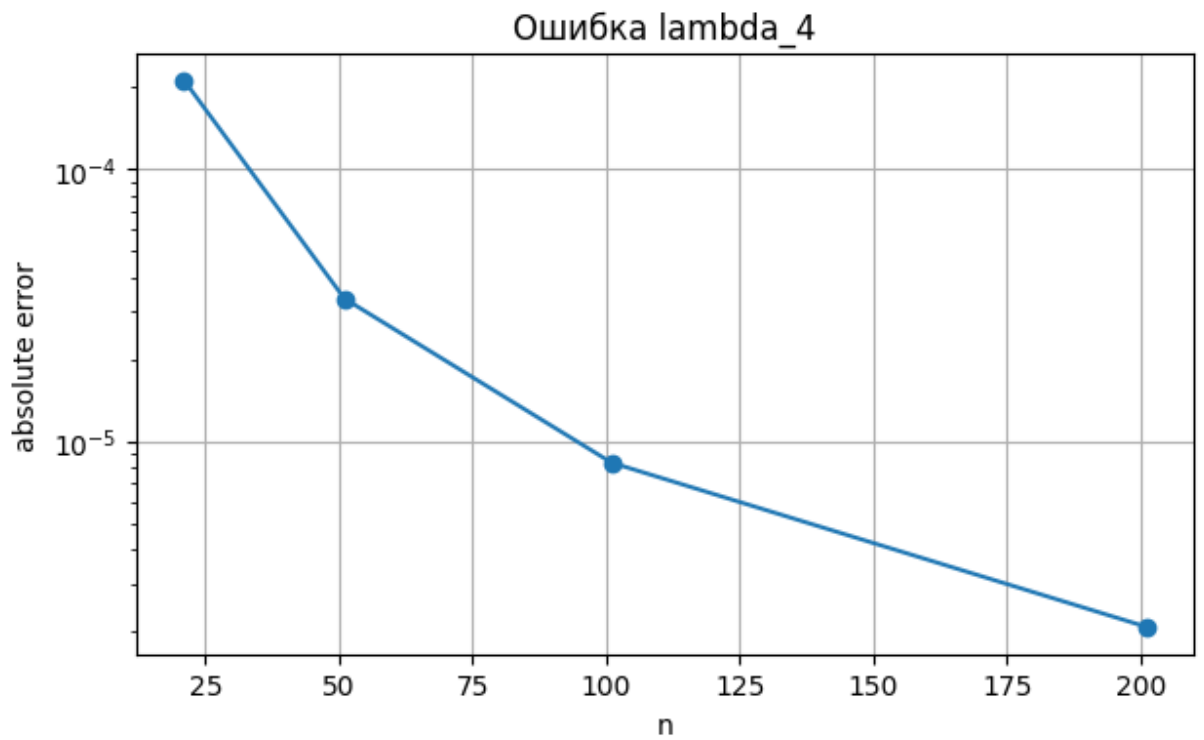
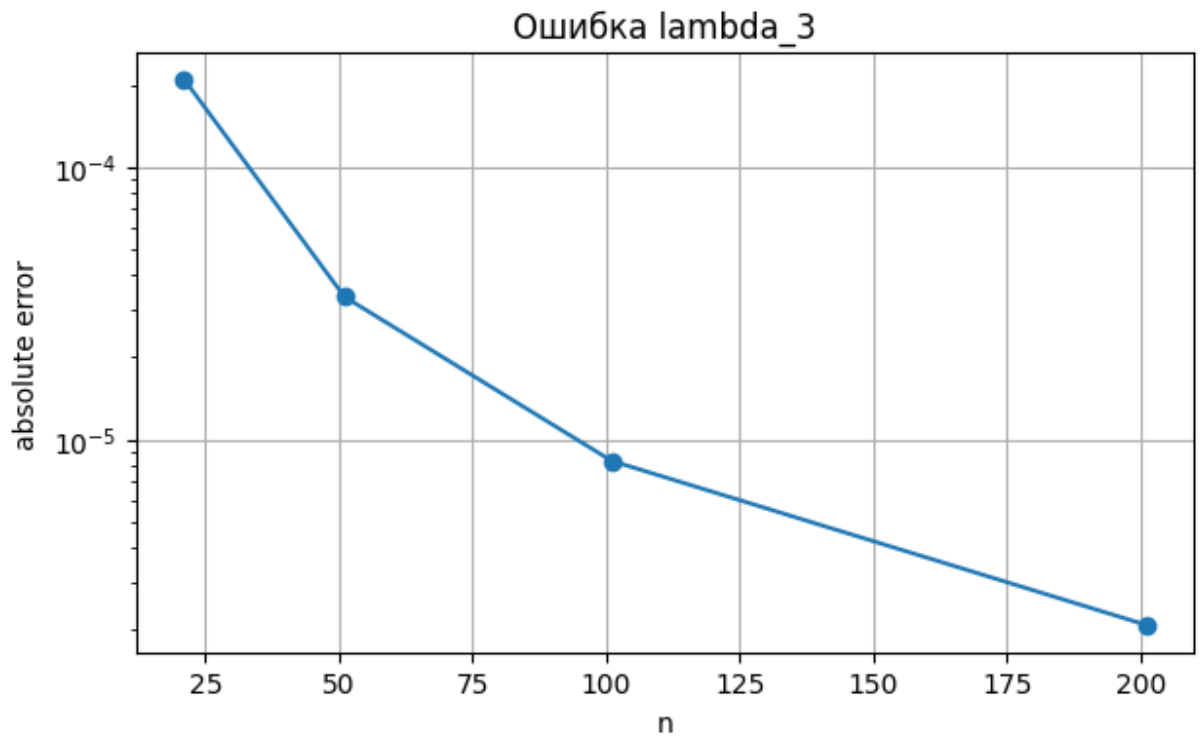
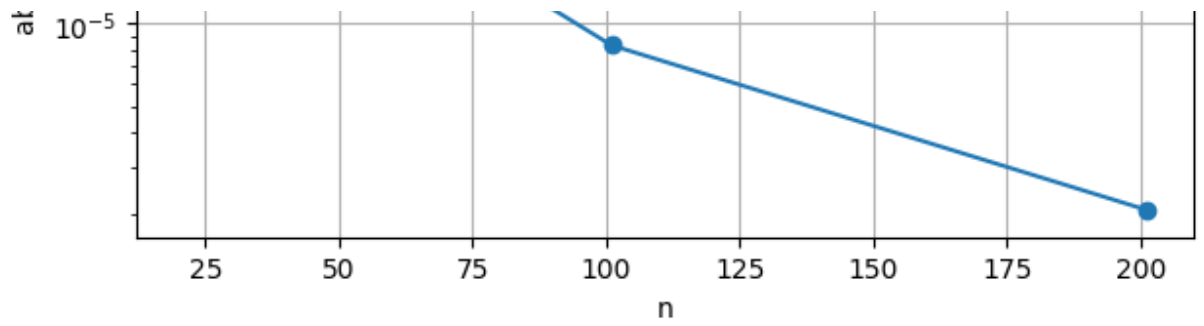
При увеличении числа узлов сетки n ошибка быстро уменьшается.
метод трапеций корректно аппроксимирует интегральный оператор, а
увеличение размера сетки повышает точность результата.

График ошибки в логарифмической шкале

Логарифмическая шкала по оси y удобна, когда ошибка уменьшается на порядки. Если график идёт вниз при росте n , значит численная схема сходится.

```
for k in range(1, 6):  
    part = err_spectrum[err_spectrum["k"] == k]  
    plt.figure(figsize=(7, 4))  
    plt.plot(part["n"], part["abs_error"], marker="o")  
    plt.yscale("log")  
    plt.title(f"Ошибка lambda_{k}")  
    plt.xlabel("n")  
    plt.ylabel("absolute error")  
    plt.grid(True)  
    plt.show()
```





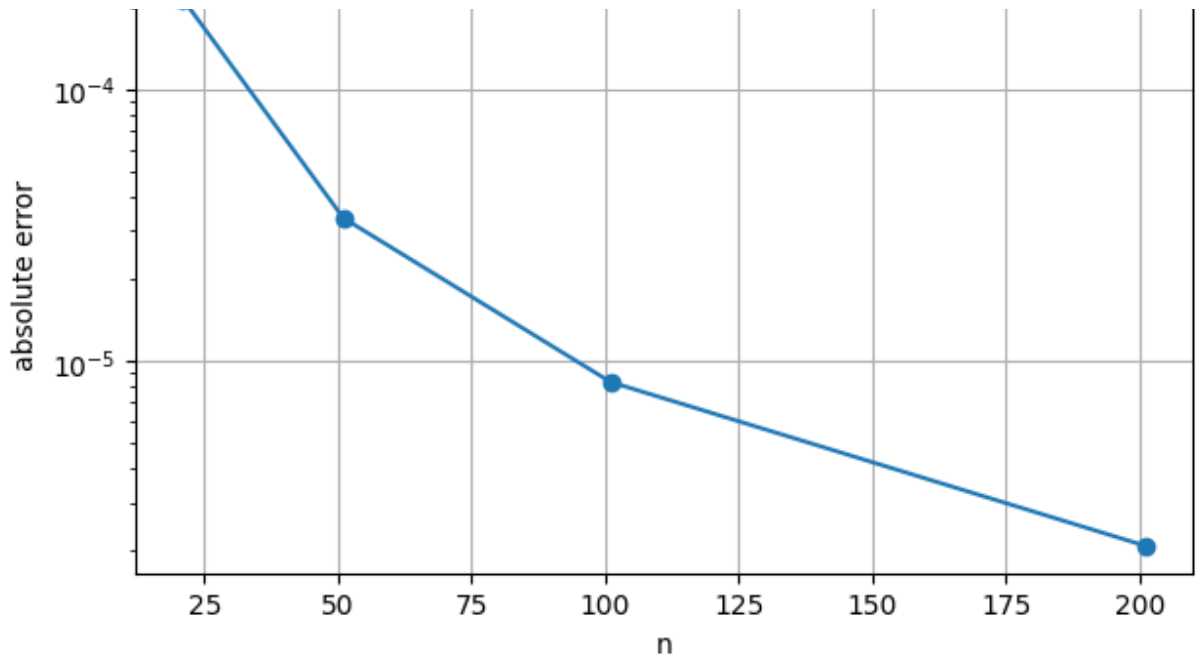


Таблица ошибок собственных функций

Кроме собственных значений важно сравнивать и собственные функции. Для этого используется дискретная L^2 -ошибка, которая учитывает отклонение всей кривой на сетке.

```
def eigenfunction_error_table(n_values, method="trapezoid", k_max=4):
    rows = []
    for n in n_values:
        x = make_uniform_grid(n)
        w = get_weights(n, method)
        _, phi = compute_discrete_spectrum(x, w)
        phi = normalize_discrete_functions(phi, w)
        phi = orient_function_signs(phi, x)
        for k in range(1, k_max + 1):
            err = eigenfunction_error(phi[:, k-1], exact_eigenfunc)
            rows.append({"method": method, "n": n, "k": k, "function": phi[:, k-1], "error": err})
    return pd.DataFrame(rows)
```

```
err_functions = eigenfunction_error_table(n_values, method="trapezc  
err_functions
```

	method	n	k	function_error
0	trapezoid	21	1	2.017059e-16
1	trapezoid	21	2	9.540008e-16
2	trapezoid	21	3	2.000000e+00
3	trapezoid	21	4	2.000000e+00
4	trapezoid	51	1	2.070071e-16
5	trapezoid	51	2	9.679958e-16
6	trapezoid	51	3	2.000000e+00
7	trapezoid	51	4	2.000000e+00
8	trapezoid	101	1	4.947524e-16
9	trapezoid	101	2	1.998812e-15
10	trapezoid	101	3	2.000000e+00
11	trapezoid	101	4	2.000000e+00
12	trapezoid	201	1	8.016764e-16
13	trapezoid	201	2	1.815160e-15
14	trapezoid	201	3	2.000000e+00
15	trapezoid	201	4	2.000000e+00

Далее: [New interactive sheet](#)

Усечённое спектральное разложение ядра

Для симметричного положительного ядра естественно использовать разложение

$$K(x, t) = \sum_{k=1}^{\infty} \lambda_k \varphi_k(x) \varphi_k(t).$$

На практике берётся конечная сумма

$$K_m(x, t) = \sum_{k=1}^m \lambda_k \varphi_k(x) \varphi_k(t).$$

```
def truncated_kernel_approximation(x, m):
    K_m = np.zeros((len(x), len(x)))
    lambdas = exact_eigenvalues(m)
    for k in range(1, m + 1):
        phi_k = exact_eigenfunction(x, k)
        K_m += lambdas[k-1] * np.outer(phi_k, phi_k)
    return K_m
```

Норма Фробениуса

Ошибка приближения ядра измеряется матричной нормой Фробениуса:

$$\|K - K_m\|_F = \left(\sum_{i,j} (K_{ij} - K_{m,ij})^2 \right)^{1/2}.$$

Чем меньше эта величина, тем лучше усечённое разложение приближает исходное ядро.

```
def frobenius_error(K_true, K_approx):
    return np.linalg.norm(K_true - K_approx, ord="fro")
```

Ошибка усечённого ядра при разных m

Здесь сравниваются приближения K_m при разных m . Чем больше собственных пар используется, тем ближе сумма к исходному ядру.


```

x_kernel = make_uniform_grid(100)
K_true = brownian_kernel(x_kernel, x_kernel)

rows = []
for m in [1, 2, 5, 10, 20, 40]:
    K_m = truncated_kernel_approximation(x_kernel, m)
    rows.append({"m": m, "frobenius_error": frobenius_error(K_true,
pd.DataFrame(rows)

```

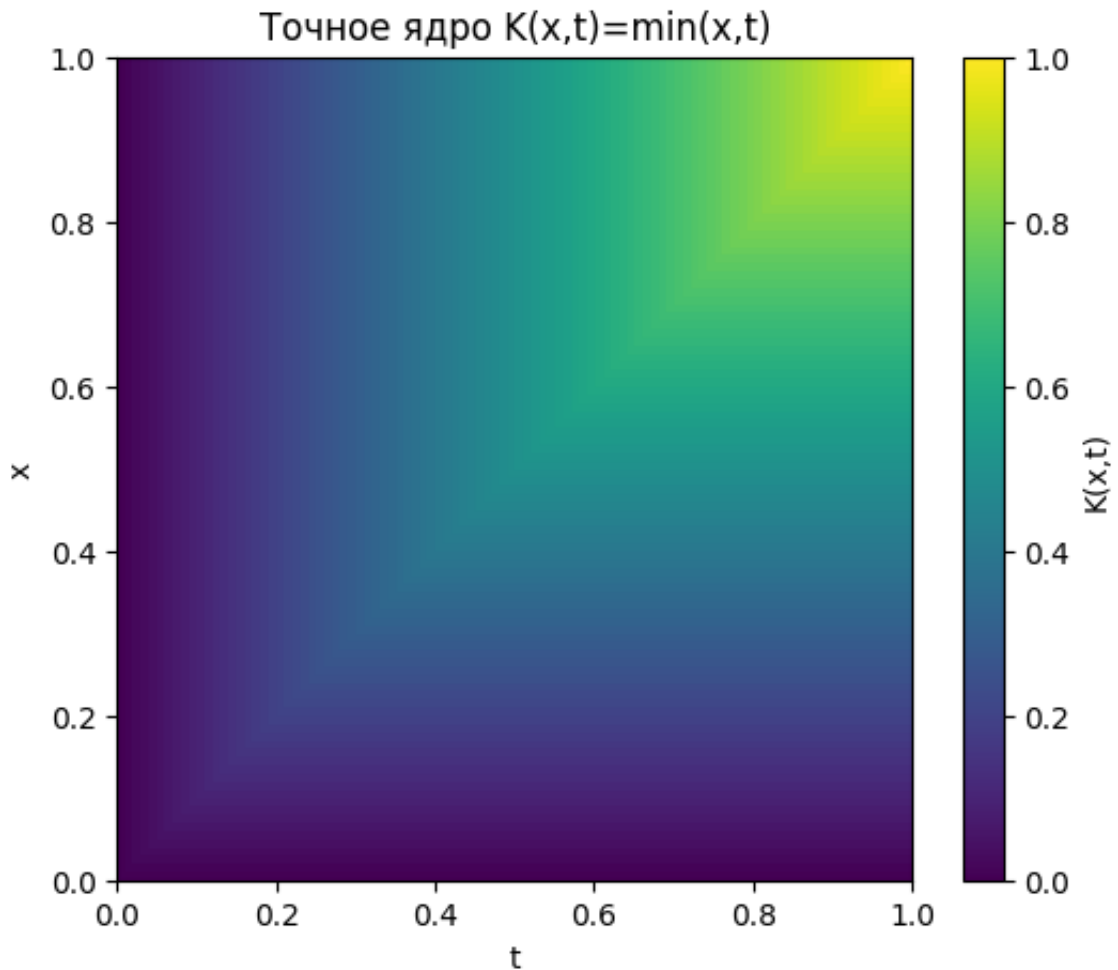
	m	frobenius_error
0	1	4.912450
1	2	1.959988
2	5	0.521801
3	10	0.189373
4	20	0.071702
5	40	0.030596

Ошибка Фробениуса уменьшается при росте m . Это означает, что добавление новых собственных пар улучшает приближение ядра. При этом основной вклад дают первые собственные значения, так как спектр убывает.

Визуализация точного ядра

Матрица ядра изображается как тепловая карта. Для $K(x, t) = \min(x, t)$ значения растут по направлению к правому верхнему углу и имеют характерную симметрию относительно главной диагонали.

```
plt.figure(figsize=(6, 5))
plt.imshow(K_true, origin="lower", extent=[0, 1, 0, 1], aspect="aut")
plt.colorbar(label="K(x,t)")
plt.title("Точное ядро  $K(x,t)=\min(x,t)$ ")
plt.xlabel("t")
plt.ylabel("x")
plt.show()
```



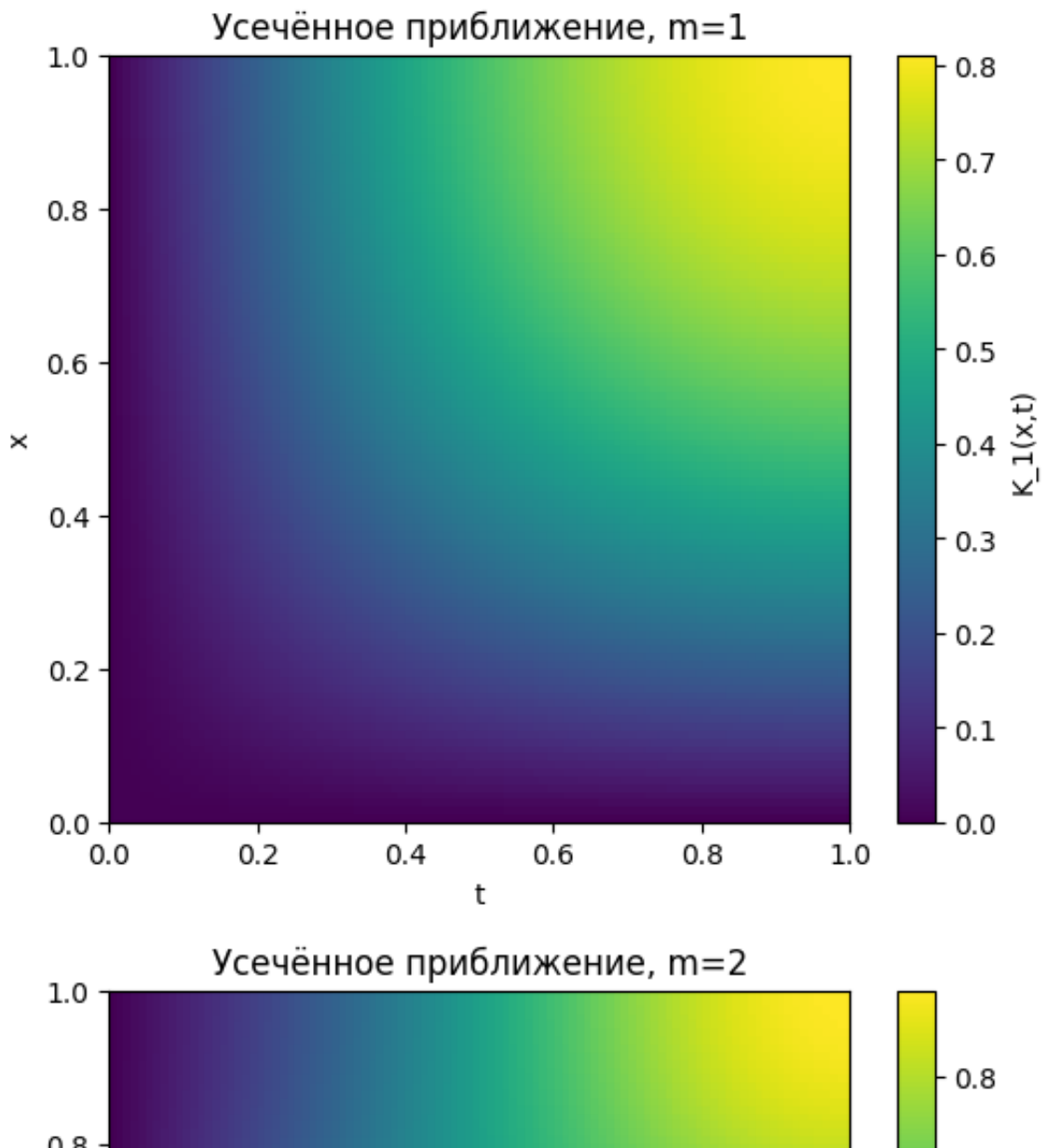
Значение ядра определяется меньшим из двух аргументов x и t . Поэтому около начала координат значения близки к нулю, а максимальные значения появляются в правом верхнем углу, где и x , и t близки к 1.

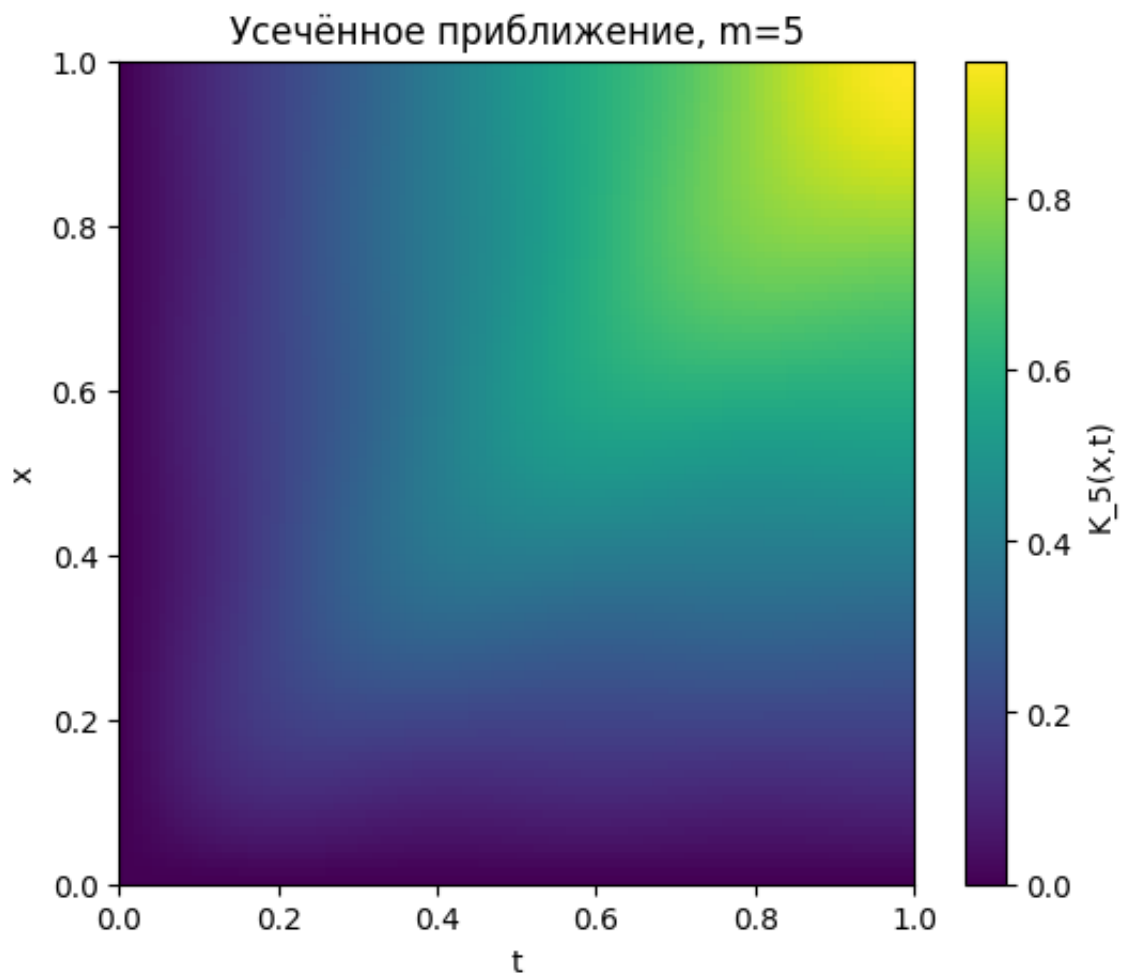
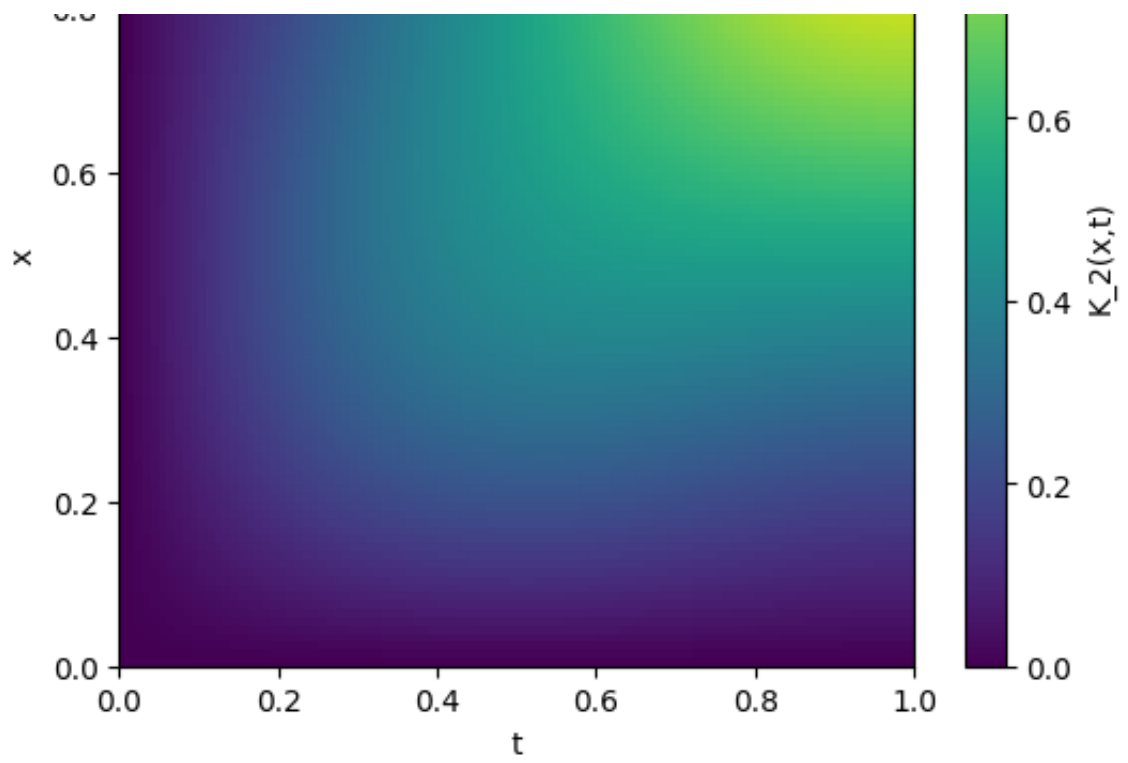
визуализация подтверждает симметричность ядра и показывает его неотрицательность на всём квадрате $[0, 1] \times [0, 1]$. Это согласуется с тем, что оператор является самосопряжённым и положительным.

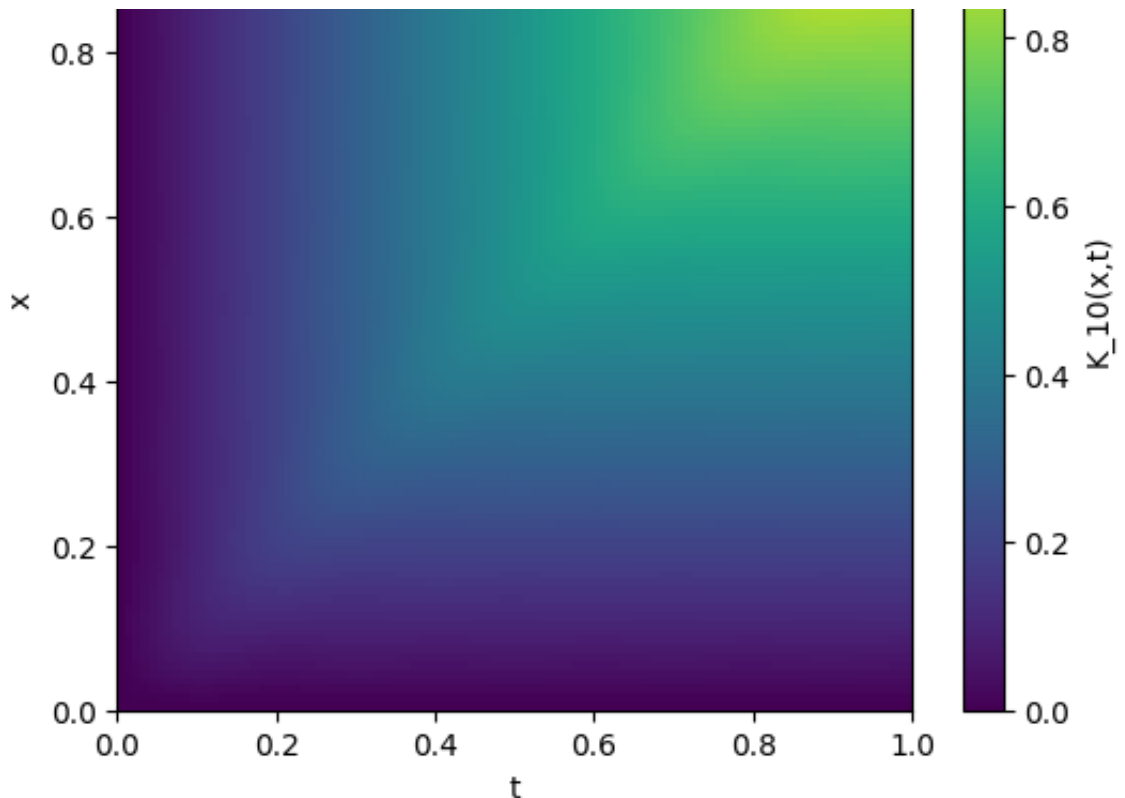
Визуализация усечённых ядер

На графиках видно, как добавление новых собственных пар постепенно восстанавливает форму точного ядра. Первые члены дают грубую структуру, последующие уточняют детали.

```
for m in [1, 2, 5, 10]:
    K_m = truncated_kernel_approximation(x_kernel, m)
    plt.figure(figsize=(6, 5))
    plt.imshow(K_m, origin="lower", extent=[0, 1, 0, 1], aspect="auto")
    plt.colorbar(label=f" $K_{\{m\}}(x,t)$ ")
    plt.title(f"Усечённое приближение, m={m}")
    plt.xlabel("t")
    plt.ylabel("x")
    plt.show()
```







Сравнение квадратурных формул

В этом блоке одновременно сравниваются методы прямоугольников, трапеций и Симпсона. Это нужно, чтобы проверить не только влияние размера сетки, но и влияние выбора формулы интегрирования.

```
quad_tables = []
for method in ["rectangle", "trapezoid", "simpson"]:
    quad_tables.append(spectrum_error_table([21, 51, 101, 201], met

quad_comparison = pd.concat(quad_tables, ignore_index=True)
quad_comparison.head(30)
```

	method	n	k	lambda_exact	lambda_num	abs_error
0	rectangle	21	1	0.405285	0.405724	0.000440
1	rectangle	21	2	0.045032	0.045257	0.000226
2	rectangle	21	3	0.016211	0.016421	0.000210
3	rectangle	51	1	0.405285	0.405357	0.000072
4	rectangle	51	2	0.045032	0.045069	0.000037
5	rectangle	51	3	0.016211	0.016246	0.000034
6	rectangle	101	1	0.405285	0.405303	0.000018



6	rectangle	101	1	0.100200	0.100000	0.000000
7	rectangle	101	2	0.045032	0.045041	0.000009
8	rectangle	101	3	0.016211	0.016220	0.000009
9	rectangle	201	1	0.405285	0.405289	0.000005
10	rectangle	201	2	0.045032	0.045034	0.000002
11	rectangle	201	3	0.016211	0.016214	0.000002
12	trapezoid	21	1	0.405285	0.405493	0.000208
13	trapezoid	21	2	0.045032	0.045241	0.000209
14	trapezoid	21	3	0.016211	0.016421	0.000210
15	trapezoid	51	1	0.405285	0.405318	0.000033
16	trapezoid	51	2	0.045032	0.045065	0.000033
17	trapezoid	51	3	0.016211	0.016245	0.000033
18	trapezoid	101	1	0.405285	0.405293	0.000008
19	trapezoid	101	2	0.045032	0.045040	0.000008
20	trapezoid	101	3	0.016211	0.016220	0.000008
21	trapezoid	201	1	0.405285	0.405287	0.000002
22	trapezoid	201	2	0.045032	0.045034	0.000002
23	trapezoid	201	3	0.016211	0.016213	0.000002
24	simpson	21	1	0.405285	0.405563	0.000278
25	simpson	21	2	0.045032	0.045312	0.000280
26	simpson	21	3	0.016211	0.016496	0.000285
27	simpson	51	1	0.405285	0.405329	0.000044
28	simpson	51	2	0.045032	0.045076	0.000045
29	simpson	51	3	0.016211	0.016256	0.000045

Далее: [New interactive sheet](#)

Вывод:

Все три метода дают близкие к аналитическим значения.

При увеличении числа узлов сетки n ошибка уменьшается, значит численное решение сходится к точному.

Лучше всего на этих данных выглядит метод трапеций: он даёт стабильную и маленькую ошибку.

Метод Симпсона не показывает явного преимущества, потому что ядро $K(x,t)=\min(x,t)$ имеет излом при $x=t$, а метод Симпсона лучше работает на более гладких функциях.

Итог: дискретизация выполнена корректно, а увеличение сетки повышает точность.

Неравномерная сетка

Равномерная сетка не является единственным вариантом. На неравномерной сетке расстояния между соседними узлами различаются, поэтому веса квадратуры нужно считать с учётом локальных расстояний.

```
def make_nonuniform_grid(n, seed=42):
    local_rng = np.random.default_rng(seed)
    inner = np.sort(local_rng.uniform(0, 1, size=n-2))
    return np.concatenate([[0.0], inner, [1.0]])
```

Веса трапеций для неравномерной сетки

Для внутренних узлов вес можно интерпретировать как половину длины соседнего интервала слева плюс половину длины соседнего интервала справа:

$$w_i = \frac{x_{i+1} - x_{i-1}}{2}.$$

```
def nonuniform_trapezoid_weights(x):
    n = len(x)
    w = np.zeros(n)
    w[0] = (x[1] - x[0]) / 2
    w[-1] = (x[-1] - x[-2]) / 2
    for i in range(1, n - 1):
        w[i] = (x[i+1] - x[i-1]) / 2
    return w
```

Проверка спектра на неравномерной сетке

Этот эксперимент показывает, что дискретизация может работать и на неравномерных узлах, если веса выбраны согласованно с сеткой.

```
x_non = make_nonuniform_grid(101, seed=RANDOM_STATE)
w_non = nonuniform_trapezoid_weights(x_non)
eig_non, _ = compute_discrete_spectrum(x_non, w_non)

pd.DataFrame({
    "k": np.arange(1, 6),
    "lambda_exact": exact_eigenvalues(5),
    "lambda_nonuniform": eig_non[:5],
    "abs_error": np.abs(eig_non[:5] - exact_eigenvalues(5)),
})
```

	k	lambda_exact	lambda_nonuniform	abs_error
0	1	0.405285	0.405322	0.000037
1	2	0.045032	0.045108	0.000076
2	3	0.016211	0.016195	0.000017
3	4	0.008271	0.008422	0.000151
4	5	0.005004	0.005060	0.000057



Возмущение ядра

Чтобы проверить устойчивость спектра, добавляется малое симметричное возмущение:

$$K_{arepsilon}(x, t) = K(x, t) + arepsilon R(x, t).$$

Чем меньше *arepsilon*, тем ближе спектр возмущённого оператора к исходному.

```
def perturb_symmetric_kernel(K, eps, seed=42):
    local_rng = np.random.default_rng(seed)
    R0 = local_rng.normal(size=K.shape)
    R = (R0 + R0.T) / 2
    return K + eps * R
```

Спектр по готовой матрице ядра

Когда ядро уже задано матрицей, снова строится симметричная матрица

$$B = W^{1/2} K W^{1/2}$$

и по ней находятся собственные значения.

```
def spectrum_from_kernel_matrix(K, w):
    sqrt_w = np.sqrt(w)
    B = sqrt_w[:, None] * K * sqrt_w[None, :]
    vals, vecs = np.linalg.eigh(B)
    order = np.argsort(vals)[::-1]
    vals = vals[order]
    vecs = vecs[:, order]
    phi = vecs / sqrt_w[:, None]
    return vals, phi
```

Эксперимент с возмущённым ядром

Здесь сравниваются первые собственные значения при разных уровнях шума *arepsilon*. Это показывает, насколько устойчивы главные компоненты спектра.

```

x = make_uniform_grid(101)
w = trapezoid_weights(101)
K = brownian_kernel(x, x)

rows = []
for eps in [0.0, 0.001, 0.005, 0.01]:
    K_eps = perturb_symmetric_kernel(K, eps=eps, seed=RANDOM_STATE)
    vals_eps, _ = spectrum_from_kernel_matrix(K_eps, w)
    for k in range(1, 4):
        rows.append({
            "eps": eps,
            "k": k,
            "lambda": vals_eps[k-1],
            "shift_from_exact": vals_eps[k-1] - exact_eigenvalues(3
        })

pd.DataFrame(rows)

```

	eps	k	lambda	shift_from_exact
0	0.000	1	0.405293	0.000008
1	0.000	2	0.045040	0.000008
2	0.000	3	0.016220	0.000008
3	0.001	1	0.405293	0.000009
4	0.001	2	0.045040	0.000008
5	0.001	3	0.016218	0.000007
6	0.005	1	0.405295	0.000010
7	0.005	2	0.045040	0.000009
8	0.005	3	0.016218	0.000007
9	0.010	1	0.405298	0.000013
10	0.010	2	0.045046	0.000015
11	0.010	3	0.016232	0.000021



Генерация винеровских траекторий

Винеровский процесс строится через независимые приращения

$$\Delta W_k \sim N(0, \Delta t_k), \quad W(t_k) = \sum_{r=1}^k \Delta W_r.$$

Его теоретическая ковариация равна

$$\mathbb{E}[W(x)W(t)] = \min(x, t).$$

```
def simulate_brownian_paths(x, n_paths, seed=42):
    local_rng = np.random.default_rng(seed)
    dt = np.diff(x)
    increments = local_rng.normal(0.0, np.sqrt(dt), size=(n_paths,
    paths = np.zeros((n_paths, len(x)))
    paths[:, 1:] = np.cumsum(increments, axis=1)
    return paths
```

Эмпирическое ковариационное ядро

По набору траекторий строится оценка ковариации

$$\widehat{K}(x_i, x_j) = \frac{1}{M} \sum_{m=1}^M W^{(m)}(x_i) W^{(m)}(x_j).$$

При увеличении числа траекторий M эта оценка должна приближаться к теоретическому ядру.

```
def empirical_covariance_kernel(paths):
    return paths.T @ paths / paths.shape[0]
```

Сравнение эмпирического и теоретического ядра

В этом эксперименте увеличивается число траекторий винеровского процесса. Ожидается, что ошибка между $\widehat{K}$ и K будет уменьшаться при росте M .

```

x = make_uniform_grid(101)
K_true = brownian_kernel(x, x)

for M_paths in [100, 1000, 5000]:
    paths = simulate_brownian_paths(x, n_paths=M_paths, seed=RANDOM)
    K_hat = empirical_covariance_kernel(paths)
    rel_err = frobenius_error(K_true, K_hat) / frobenius_error(K_true)
    print(f"M={M_paths:5d} | относительная ошибка Фробениуса = {rel_err:5f}")

```

```

M= 100 | относительная ошибка Фробениуса = 0.0474
M= 1000 | относительная ошибка Фробениуса = 0.0538
M= 5000 | относительная ошибка Фробениуса = 0.0080

```

При увеличении числа траекторий эмпирическая ковариация становится ближе к теоретическому ядру $\min(x, t)$. Это подтверждает связь рассматриваемого ядра с ковариационной функцией винеровского процесса.

Вывод по кейсу 1

Матрица $G_{ij} = \min(x_i, x_j)$ симметрична. Собственные значения матрицы Грама неотрицательны с точностью до вычислительной ошибки. Численный спектр оператора приближает аналитические значения. Первые собственные функции совпадают с аналитическими синусами. При увеличении сетки ошибка уменьшается. Усечённое спектральное разложение лучше приближает ядро при росте m . Эмпирическая ковариация винеровского процесса приближает $\min(x, t)$.

✓ Кейс 7: аддитивная модель

В регрессионной задаче наблюдается зависимость

$$y = f(x_1, \dots, x_p) + \varepsilon,$$

где ε — случайный шум. Полная многомерная функция f может быть сложной, поэтому её трудно восстанавливать напрямую.

Аддитивная модель использует более простое представление:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p g_j(x_j).$$

То есть каждый признак вносит отдельный одномерный вклад. Это упрощает задачу, уменьшает число параметров и делает модель интерпретируемой.

Каждая компонента раскладывается по базису:

$$g_j(t) = \sum_{m=1}^M a_{jm} \psi_m(t).$$

Тогда вся модель становится линейной по параметрам:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p \sum_{m=1}^M a_{jm} \psi_m(x_j).$$

Именно поэтому её можно обучать методом наименьших квадратов или ridge-регрессией.

Генерация синтетических аддитивных данных

В кейсе 7 рассматривается регрессионная задача

$$y = f(x_1, \dots, x_p) + \text{arepsilon}, \quad x_j \in [0, 1].$$

Для аддитивного эксперимента функция имеет вид

$$f(x) = g_1(x_1) + g_2(x_2) + g_3(x_3),$$

где

$$g_1(t) = \sin(2\pi t), \quad g_2(t) = 0.5(t - 0.5)^2, \quad g_3(t) = e^{-3t}.$$

```
def generate_additive_data(n_samples=1000, noise_std=0.1, seed=42):
    local_rng = np.random.default_rng(seed)
    X = local_rng.uniform(0, 1, size=(n_samples, 3))

    y_true = (
        np.sin(2 * np.pi * X[:, 0])
        + 0.5 * (X[:, 1] - 0.5) ** 2
        + np.exp(-3 * X[:, 2])
    )

    y = y_true + local_rng.normal(0, noise_std, size=n_samples)
    return X, y, y_true
```

Истинная первая компонента

Первая компонента задаёт периодическую зависимость:

$$g_1(t) = \sin(2\pi t).$$

Она нелинейная и не может быть хорошо описана обычной линейной функцией βt .

```
def true_g1(t):
    return np.sin(2 * np.pi * t)
```

Истинная вторая компонента

Вторая компонента квадратичная:

$$g_2(t) = 0.5(t - 0.5)^2.$$

Она показывает, что вклад признака может быть U-образным, а не монотонным.

```
def true_g2(t):
    return 0.5 * (t - 0.5) ** 2
```

Истинная третья компонента

Третья компонента экспоненциально убывает:

$$g_3(t) = e^{-3t}.$$

Такая форма хорошо показывает пользу гибкого базисного разложения.

```
def true_g3(t):
    return np.exp(-3 * t)
```

Train/test split

```
X, y, y_true = generate_additive_data(n_samples=1200, noise_std=0.1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=RANDOM_STATE
)

print(X_train.shape, X_test.shape)
```

(900, 3) (300, 3)

Нормировка признаков в $[0, 1]$

$$x'_{ij} = \frac{x_{ij} - \min(x_j)}{\max(x_j) - \min(x_j)}.$$

Для синтетических данных признаки уже лежат в этом диапазоне, но функция нужна и для реальных данных.

```
def scale_to_unit_interval(X_train, X_test):
    scaler = MinMaxScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)
    return X_train_s, X_test_s, scaler
```

```
X_train_s, X_test_s, scaler = scale_to_unit_interval(X_train, X_test)

print("min train:", X_train_s.min(axis=0))
print("max train:", X_train_s.max(axis=0))
```

min train: [0. 0. 0.]
max train: [1. 1. 1.]

Полиномиальный базис

Каждая одномерная функция $g_j(t)$ приближается через конечный базис:

$$g_j(t) = \sum_{m=1}^M a_{jm} \psi_m(t).$$

Для полиномиального базиса

$$\psi_m(t) = t^m, \quad m = 1, \dots, M.$$

```
def polynomial_basis_1d(t, M):
    return np.column_stack([t ** power for power in range(1, M + 1)])
```

Тригонометрический базис

Тригонометрический базис удобен для периодических зависимостей:

$$\psi_{2k-1}(t) = \sin(2\pi kt), \quad \psi_{2k}(t) = \cos(2\pi kt).$$

Такой базис особенно хорошо подходит для компоненты $\sin(2\pi x_1)$.

```
def trigonometric_basis_1d(t, M):
    cols = []
    for k in range(1, M + 1):
        cols.append(np.sin(2 * np.pi * k * t))
        cols.append(np.cos(2 * np.pi * k * t))
    return np.column_stack(cols)
```

Выбор одномерного базиса

Эта функция позволяет переключаться между полиномиальным и тригонометрическим базисом без переписывания всей модели.

```
def build_1d_basis(t, M, basis_type):
    if basis_type == "polynomial":
        return polynomial_basis_1d(t, M)
    if basis_type == "trigonometric":
        return trigonometric_basis_1d(t, M)
    raise ValueError("Неизвестный basis_type")
```


Блочная матрица признаков аддитивной модели

После базисного разложения модель записывается как линейная по параметрам:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p \sum_{m=1}^M a_{jm} \psi_m(x_j).$$

Для одного объекта строка матрицы признаков имеет вид

$$z_i = (\psi_1(x_{i1}), \dots, \psi_M(x_{i1}), \dots, \psi_1(x_{ip}), \dots, \psi_M(x_{ip})).$$

Тогда задача становится обычной регрессией

$$\hat{y}_i = \beta_0 + z_i^T \theta.$$

```
def build_additive_design_matrix(X, M, basis_type="polynomial"):
    blocks = []
    for j in range(X.shape[1]):
        blocks.append(build_1d_basis(X[:, j], M=M, basis_type=basis_type))
    return np.hstack(blocks)
```

Метрики качества регрессии

Используются три стандартные метрики:

$$MSE = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2,$$

$$RMSE = \sqrt{MSE},$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

Чем меньше MSE и RMSE, тем лучше. Чем ближе R^2 к 1, тем лучше модель объясняет данные.

```
def regression_metrics(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    return {"MSE": mse, "RMSE": rmse, "R2": r2}
```

Обучение аддитивной модели по МНК

Параметры модели находятся из задачи наименьших квадратов:

$$Q(\theta) = \sum_{i=1}^n (y_i - \beta_0 - z_i^T \theta)^2 \rightarrow \min_{\theta}.$$

Так как модель линейна по параметрам, её можно обучать обычной

`LinearRegression`.

```
def fit_additive_ols(X_train, y_train, M, basis_type="polynomial"):
    Z_train = build_additive_design_matrix(X_train, M=M, basis_type=
    model = LinearRegression()
    model.fit(Z_train, y_train)
    return model
```

Предсказание аддитивной модели

Для новых объектов строится такая же блочная матрица Z , после чего предсказание получается по формуле

$$\hat{y} = \beta_0 + Z\theta.$$

```
def predict_additive(model, X, M, basis_type="polynomial"):
    Z = build_additive_design_matrix(X, M=M, basis_type=basis_type)
    return model.predict(Z)
```

Первая аддитивная модель

Здесь используется полиномиальный базис степени $M = 6$. Это уже более гибкая модель, чем линейная регрессия, потому что каждый признак может входить нелинейно.

```

M = 6
basis_type = "polynomial"

add_model = fit_additive_ols(X_train_s, y_train, M=M, basis_type=basis_type)

y_pred_train_add = predict_additive(add_model, X_train_s, M=M, basis_type=basis_type)
y_pred_test_add = predict_additive(add_model, X_test_s, M=M, basis_type=basis_type)

print("Train:", regression_metrics(y_train, y_pred_train_add))
print("Test:", regression_metrics(y_test, y_pred_test_add))

```

```

Train: {'MSE': 0.00915165521255636, 'RMSE': np.float64(0.09566428389)}
Test: {'MSE': 0.011268374516952553, 'RMSE': np.float64(0.10615260014)}

```

Модель работает хорошо: ошибка маленькая, а R^2 около 0.98.

Результаты на train и test близкие, значит сильного переобучения нет.

Итог: аддитивная модель хорошо описывает данные и уверенно обобщается на тестовую выборку.

Ridge-регрессия для аддитивной модели

Ridge добавляет штраф на величину коэффициентов:

$$Q_{\text{ridge}}(\theta) = \sum_i (y_i - \hat{y}_i)^2 + \alpha \sum_r \theta_r^2.$$

Это снижает риск переобучения, особенно при большом числе базисных функций.

```

def fit_additive_ridge(X_train, y_train, M, alpha=1.0, basis_type="polynomial"):
    Z_train = build_additive_design_matrix(X_train, M=M, basis_type=basis_type)
    model = Ridge(alpha=alpha)
    model.fit(Z_train, y_train)
    return model

```

Проверка ridge-модели

В этом блоке обучается аддитивная ridge-модель с большим числом полиномиальных функций. Сравнение с обычной МНК-моделью показывает, помогает ли регуляризация удерживать качество на тесте.

```
ridge_add_model = fit_additive_ridge(X_train_s, y_train, M=12, alph
y_pred_test_ridge = predict_additive(ridge_add_model, X_test_s, M=1
regression_metrics(y_test, y_pred_test_ridge)
```

```
{'MSE': 0.07247522263634674,
 'RMSE': np.float64(0.2692122260157342),
 'R2': 0.8762965508695523}
```

Качество модели хуже, чем у аддитивной модели.

Модель объясняет около 87.6% изменчивости целевой переменной, что в целом неплохо, но ошибка заметно выше.

Итог: эта модель улавливает основную зависимость, но описывает данные менее точно, чем аддитивная модель.

Размеры блоков коэффициентов

У каждого признака есть свой блок коэффициентов. Например, если признаков $p = 3$, а на каждый берётся $M = 6$ функций, то всего будет 18 базисных коэффициентов плюс свободный член.

```
def get_basis_block_sizes(X, M, basis_type):
    sizes = []
    for j in range(X.shape[1]):
        sizes.append(build_1d_basis(X[:, j], M=M, basis_type=basis_
    return sizes
```

Разделение коэффициентов по признакам

Чтобы восстановить отдельные функции $g_j(t)$, нужно понять, какие коэффициенты относятся к какому признаку. Поэтому общий вектор коэффициентов делится на блоки.

```
def split_coefficients_by_feature(coef, block_sizes):
    blocks = []
    start = 0
    for size in block_sizes:
        blocks.append(coef[start:start + size])
        start += size
    return blocks
```

Восстановление компонент модели

Для каждого признака вычисляется вклад

$$\hat{g}_j(x_{ij}) = \sum_{m=1}^M \hat{a}_{jm} \psi_m(x_{ij}).$$

Сумма всех компонент вместе со свободным членом даёт итоговое предсказание.

```
def additive_components(model, X, M, basis_type):
    block_sizes = get_basis_block_sizes(X, M, basis_type)
    coef_blocks = split_coefficients_by_feature(model.coef_, block_

    components = []
    for j in range(X.shape[1]):
        B_j = build_1d_basis(X[:, j], M=M, basis_type=basis_type)
        components.append(B_j @ coef_blocks[j])

    return np.column_stack(components)
```

Центрирование компонент

Аддитивное разложение не единственно: константу можно перенести из одной компоненты в другую. Для однозначности вводят условие

$$\sum_{i=1}^n \hat{g}_j(x_{ij}) = 0.$$

После центрирования средние компонент переносятся в свободный член.

```
def centered_additive_components(model, X_train, X_eval, M, basis_t
    comp_train = additive_components(model, X_train, M, basis_type)
    comp_eval = additive_components(model, X_eval, M, basis_type)

    means = comp_train.mean(axis=0)
    comp_eval_centered = comp_eval - means[None, :]
    intercept_centered = model.intercept_ + means.sum()

    return intercept_centered, comp_eval_centered, means
```

Проверка центрирования

После центрирования среднее каждой компоненты на train-выборке должно быть близко к нулю. Свободный член при этом корректируется так, чтобы предсказания модели не изменились.

```

intercept_c, comp_train_c, comp_means = centered_additive_component
    add_model, X_train_s, X_train_s, M=M, basis_type=basis_type
)

print("Средние после центрирования:", comp_train_c.mean(axis=0))
print("Старый intercept:", add_model.intercept_)
print("Новый intercept:", intercept_c)

pred_centered = intercept_c + comp_train_c.sum(axis=1)
print("Максимальная разница предсказаний:", np.max(np.abs(pred_cent

```

```

Средние после центрирования: [-0.  0. -0.]
Старый intercept: 1.1089926443932254
Новый intercept: 0.3665096205196863
Максимальная разница предсказаний: 2.90878432451791e-14

```

Средние значения компонент после центрирования близки к нулю. Значит, неоднозначность разложения устранена: постоянная часть перенесена в свободный член, а отдельные функции g_j интерпретируются как чистые вклады признаков.

центрирование изменило только вид разложения, но сами предсказания модели не изменились.

Линейная модель как базовый уровень

Линейная регрессия имеет вид

$$\hat{y} = \beta_0 + \sum_{j=1}^p \beta_j x_j.$$

Она используется как простая базовая модель для сравнения с аддитивным подходом.

```
def fit_linear_model(X_train, y_train):  
    model = LinearRegression()  
    model.fit(X_train, y_train)  
    return model
```

Если зависимость действительно нелинейная, линейная модель должна проигрывать аддитивной. Это особенно ожидаемо из-за синусоидальной и экспоненциальной компонент.

```
lin_model = fit_linear_model(X_train_s, y_train)  
  
y_pred_train_lin = lin_model.predict(X_train_s)  
y_pred_test_lin = lin_model.predict(X_test_s)  
  
print("Train:", regression_metrics(y_train, y_pred_train_lin))  
print("Test:", regression_metrics(y_test, y_pred_test_lin))  
  
Train: {'MSE': 0.20756277164239237, 'RMSE': np.float64(0.45559057457)  
Test: {'MSE': 0.20608246926593712, 'RMSE': np.float64(0.453963070376
```

Линейная модель показала качество хуже, чем аддитивная.

$R^2 \approx 0.65$, то есть модель объясняет около 65% зависимости.

Ошибки на train и test почти одинаковые, поэтому переобучения нет.

Итог: линейной модели недостаточно, потому что зависимость в данных нелинейная.

Полная полиномиальная модель

Полная модель с взаимодействиями может содержать не только отдельные степени признаков, но и произведения:

$$\hat{y} = \beta_0 + \sum_j \beta_j x_j + \sum_{j \leq k} \beta_{jk} x_j x_k + \dots$$

Она гибче аддитивной модели, но имеет больше параметров и хуже интерпретируется.

```
def fit_full_polynomial_model(X_train, y_train, degree=2):
    poly = PolynomialFeatures(degree=degree, include_bias=False)
    Z_train = poly.fit_transform(X_train)
    model = LinearRegression()
    model.fit(Z_train, y_train)
    return poly, model
```

Предсказание полной полиномиальной модели

Для тестовых данных нужно применить то же полиномиальное преобразование, которое было построено на обучающей выборке.

```
def predict_full_polynomial(poly, model, X):
    return model.predict(poly.transform(X))
```

Обучение полной полиномиальной модели

Эта модель используется как более сложный конкурент аддитивной модели. Если данные аддитивные, то взаимодействия могут быть лишними и не обязательно дадут заметный выигрыш.

```
poly, full_model = fit_full_polynomial_model(X_train_s, y_train, de
y_pred_train_full = predict_full_polynomial(poly, full_model, X_tra
y_pred_test_full = predict_full_polynomial(poly, full_model, X_test

print("Train:", regression_metrics(y_train, y_pred_train_full))
print("Test:", regression_metrics(y_test, y_pred_test_full))
print("Число признаков полной модели:", poly.transform(X_train_s).s

Train: {'MSE': 0.01318899840579679, 'RMSE': np.float64(0.11484336465
Test: {'MSE': 0.016104026679467705, 'RMSE': np.float64(0.12690164175
Число признаков полной модели: 19
```

Полная полиномиальная модель работает хорошо: R^2 на тесте около 0.97.

Но качество чуть хуже, чем у аддитивной модели, при этом признаков больше — 19.

Итог: усложнение модели не дало выигрыша, поэтому для этих данных аддитивная модель лучше и проще.

Сравнительная таблица моделей

В таблице сравниваются линейная, аддитивная и полная полиномиальная модели. Важно смотреть одновременно на качество и число параметров: модель с большим числом признаков может быть точнее, но менее устойчивой и менее интерпретируемой.

```
rows = []
models_for_table = [
    ("Linear", y_pred_train_lin, y_pred_test_lin, X_train_s.shape[1],
     "Additive polynomial M=6", y_pred_train_add, y_pred_test_add,
     "Additive ridge M=12", ridge_add_model.predict(build_additive_
     "Full polynomial degree=3", y_pred_train_full, y_pred_test_full
    ]

for name, pred_train, pred_test, n_features in models_for_table:
    tr = regression_metrics(y_train, pred_train)
    te = regression_metrics(y_test, pred_test)
    rows.append({
        "model": name,
        "n_features": n_features,
        "train_RMSE": tr["RMSE"],
        "test_RMSE": te["RMSE"],
        "train_R2": tr["R2"],
        "test_R2": te["R2"],
    })

pd.DataFrame(rows)
```

	model	n_features	train_RMSE	test_RMSE	train_R2	test_R2
0	Linear	3	0.455591	0.453963	0.649149	0.648251
1	Additive polynomial M=6	18	0.095664	0.106153	0.984531	0.980767
	Additive					

Лучше всего сработала аддитивная модель с $M=6$: у неё минимальная ошибка на тесте и самый высокий $R^2 = 0.981$.

Линейная модель слишком простая и плохо описывает нелинейную зависимость.

Полная полиномиальная модель тоже работает хорошо, но уступает аддитивной.

Ridge с $M=12$ оказался хуже, вероятно, из-за слишком сильной регуляризации. для этих данных оптимальный вариант — аддитивная полиномиальная модель с $M=6$.

Эксперимент по числу базисных функций

```
def additive_complexity_experiment(X_train, y_train, X_test, y_test):
    rows = []
    for M in M_values:
        if alpha is None:
            model = fit_additive_ols(X_train, y_train, M=M, basis_type=basis_type)
        else:
            model = fit_additive_ridge(X_train, y_train, M=M, alpha=alpha)

        pred_train = predict_additive(model, X_train, M=M, basis_type=basis_type)
        pred_test = predict_additive(model, X_test, M=M, basis_type=basis_type)
        tr = regression_metrics(y_train, pred_train)
        te = regression_metrics(y_test, pred_test)

        rows.append({
            "M": M,
            "basis_type": basis_type,
            "alpha": alpha,
            "n_features": build_additive_design_matrix(X_train, M, basis_type),
            "train_RMSE": tr["RMSE"],
            "test_RMSE": te["RMSE"],
            "train_R2": tr["R2"],
            "test_R2": te["R2"],
        })
    return pd.DataFrame(rows)
```

Запуск эксперимента для полиномиального базиса

Ожидается, что train-ошибка будет уменьшаться при росте M . Test-ошибка сначала может уменьшаться, а затем стабилизироваться или расти, если модель начинает переобучаться.

```
complexity_poly = additive_complexity_experiment(
    X_train_s, y_train, X_test_s, y_test,
    M_values=list(range(1, 21)),
    basis_type="polynomial",
    alpha=None,
)

complexity_poly.head()
```

	M	basis_type	alpha	n_features	train_RMSE	test_RMSE	train_R2
0	1	polynomial	None	3	0.455591	0.453963	0.649149
1	2	polynomial	None	6	0.433771	0.453381	0.681950
2	3	polynomial	None	9	0.115051	0.126835	0.977626
3	4	polynomial	None	12	0.114819	0.127245	0.977716
4	5	polynomial	None	15	0.095799	0.106328	0.984487

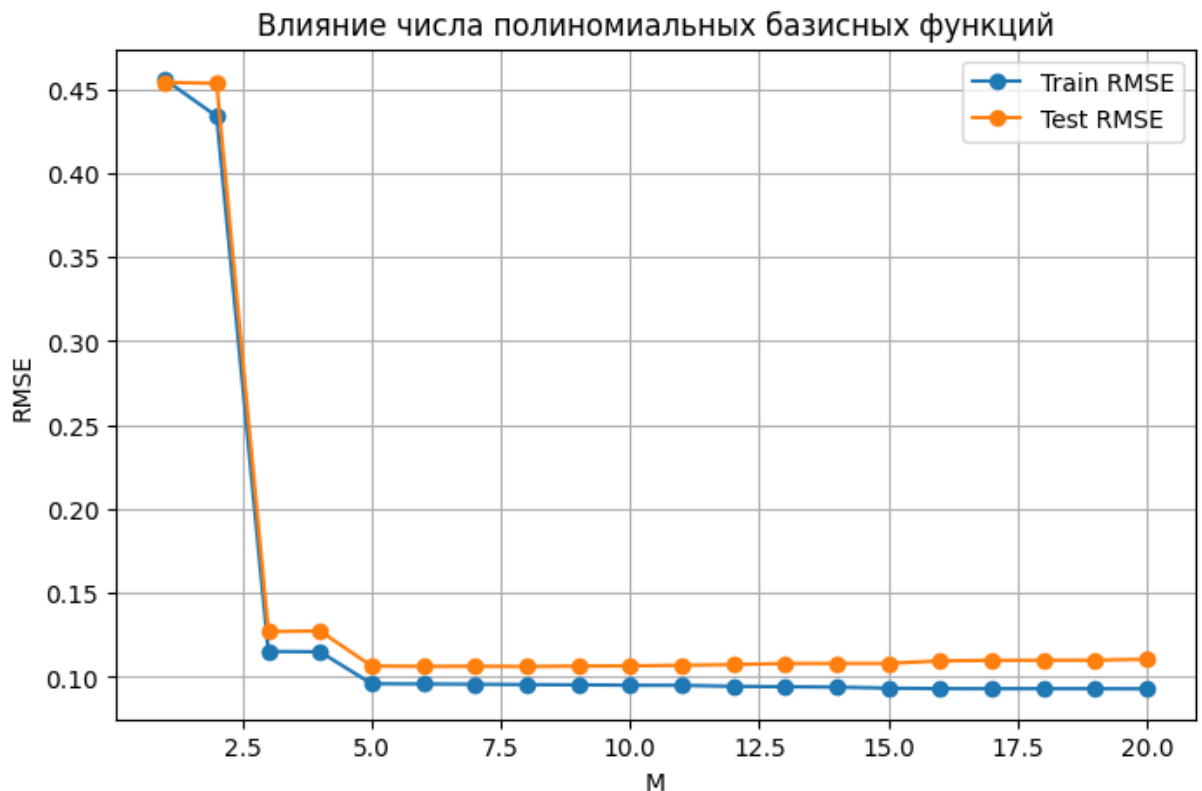
При увеличении M качество сначала улучшается.

разумный диапазон сложности — примерно $M=3-5$.

График влияния сложности модели

Если train RMSE уменьшается, а test RMSE растёт, это признак переобучения. Разумный диапазон M выбирается там, где test RMSE минимальна или почти минимальна.

```
plt.figure(figsize=(8, 5))
plt.plot(complexity_poly["M"], complexity_poly["train_RMSE"], marker=
plt.plot(complexity_poly["M"], complexity_poly["test_RMSE"], marker=
plt.title("Влияние числа полиномиальных базисных функций")
plt.xlabel("M")
plt.ylabel("RMSE")
plt.grid(True)
plt.legend()
plt.show()
```



После $M=3$ ошибка резко падает, значит модель начинает хорошо улавливать нелинейность.

Дальше, примерно после $M=5$, качество почти не улучшается: test RMSE остаётся почти на одном уровне.

Итог: увеличивать M дальше нет смысла, разумный выбор — $M=5$ или около него.

Эксперимент для тригонометрического базиса

Тригонометрический базис особенно полезен для периодических компонент. Поэтому здесь проверяется, может ли он дать более хорошее качество при меньшем числе базисных функций.

```
complexity_trig = additive_complexity_experiment(
    X_train_s, y_train, X_test_s, y_test,
    M_values=list(range(1, 8)),
    basis_type="trigonometric",
    alpha=None,
)
```

complexity_trig

	M	basis_type	alpha	n_features	train_RMSE	test_RMSE	train_R2
0	1	trigonometric	None	6	0.200481	0.189334	0.932061
1	2	trigonometric	None	12	0.171150	0.162153	0.950486
2	3	trigonometric	None	18	0.154336	0.153648	0.959737
3	4	trigonometric	None	24	0.145209	0.146374	0.964358
4	5	trigonometric	None	30	0.136463	0.143369	0.968522
5	6	trigonometric	None	36	0.131843	0.142553	0.970618
6	7	trigonometric	None	42	0.128023	0.141646	0.972295

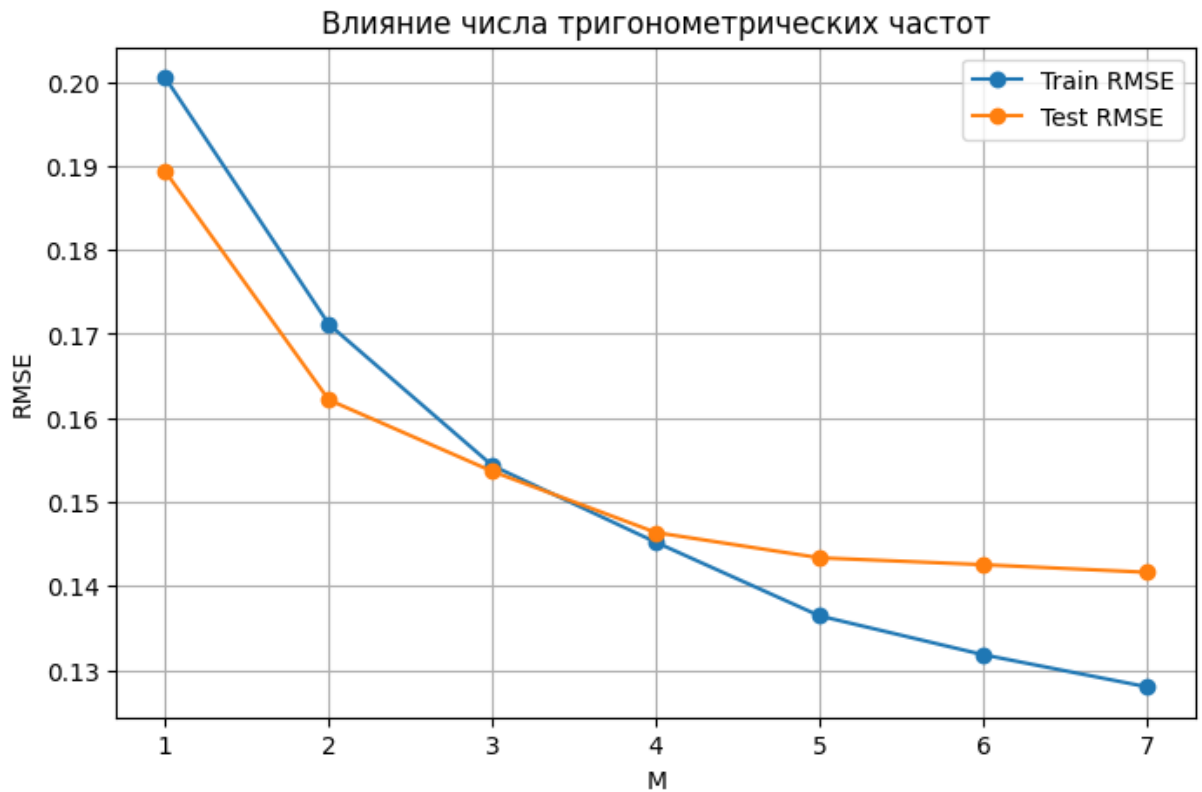
Лучший результат здесь при $M=7$: test RMSE =0.142.

тригонометрический базис работает стабильно, но хуже полиномиального базиса для этих данных.

График тригонометрического базиса

По графику можно определить, насколько быстро модель достигает хорошего качества. Если качество стабилизируется при малом M , дальнейшее увеличение сложности не нужно.

```
plt.figure(figsize=(8, 5))
plt.plot(complexity_trig["M"], complexity_trig["train_RMSE"], marker="o", color="blue")
plt.plot(complexity_trig["M"], complexity_trig["test_RMSE"], marker="o", color="orange")
plt.title("Влияние числа тригонометрических частот")
plt.xlabel("M")
plt.ylabel("RMSE")
plt.grid(True)
plt.legend()
plt.show()
```



Восстановление компоненты на сетке

Чтобы построить график $\hat{g}_j(t)$, создаётся искусственная сетка значений $t \in [0, 1]$. Остальные признаки в этот момент не важны, потому что в аддитивной модели вклад каждого признака считается отдельно.

```
def evaluate_component_on_grid(model, feature_index, t_grid, M, bas
    X_dummy = np.zeros((len(t_grid), n_features))
    X_dummy[:, feature_index] = t_grid
    comps = additive_components(model, X_dummy, M=M, basis_type=bas
    return comps[:, feature_index]
```

Центрирование кривой компоненты

При визуализации компонента тоже центрируется так же, как на train-выборке. Это делает графики сопоставимыми и устраняет произвольный вертикальный сдвиг.

```
def center_curve_by_train_component(curve, component_mean):
    return curve - component_mean
```

Графики восстановленных компонент

На графиках сравниваются истинные функции $g_j(t)$ и восстановленные функции $\hat{g}_j(t)$. Хорошее совпадение означает, что модель не только предсказывает y , но и правильно восстанавливает структуру зависимости.

```
t_grid = np.linspace(0, 1, 300)
true_functions = [true_g1, true_g2, true_g3]

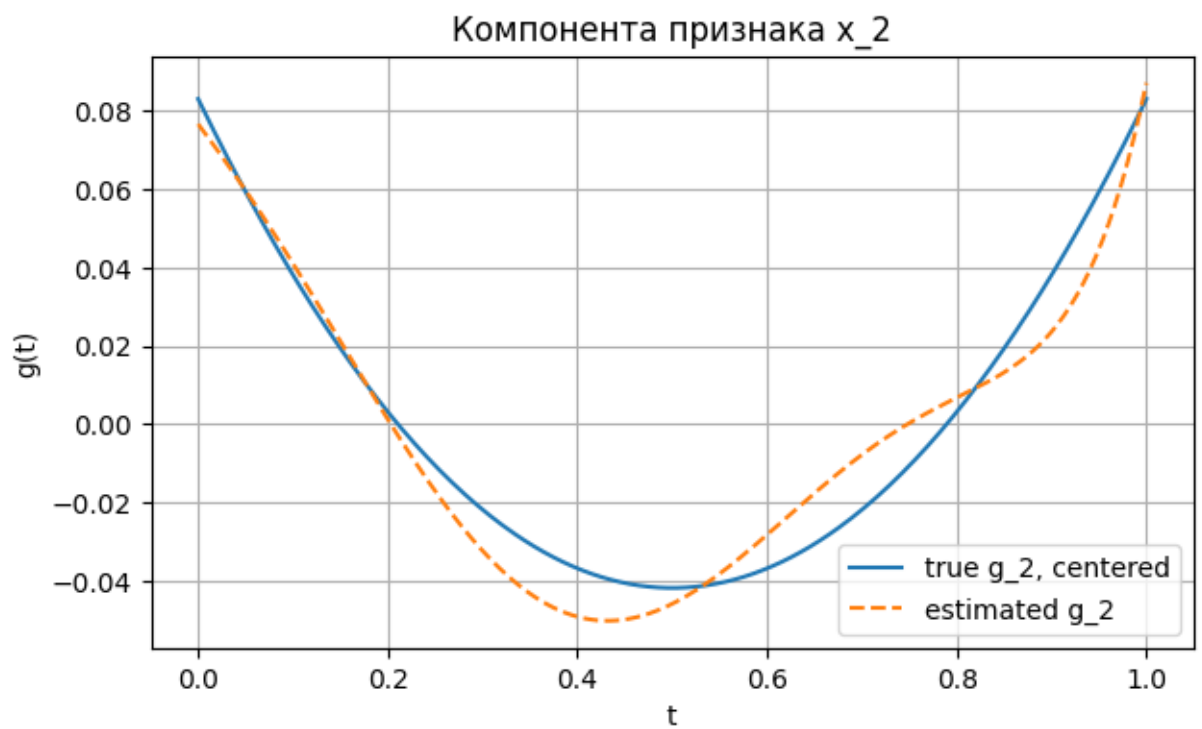
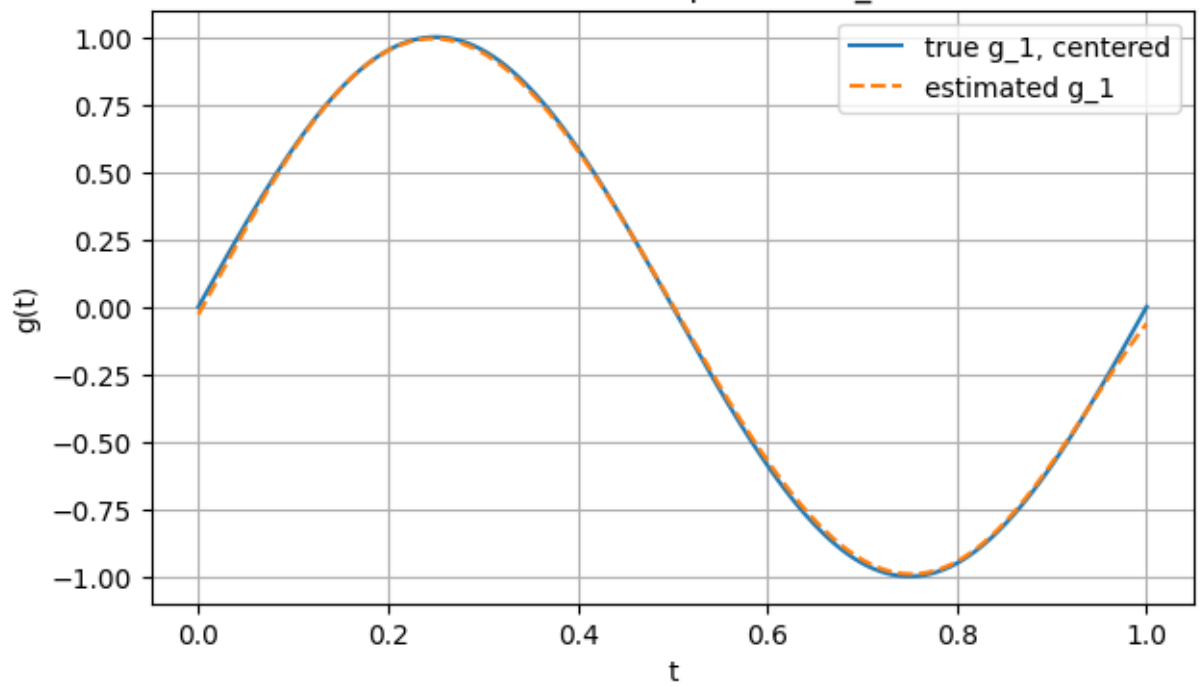
intercept_c, _, comp_means = centered_additive_components(add_model,

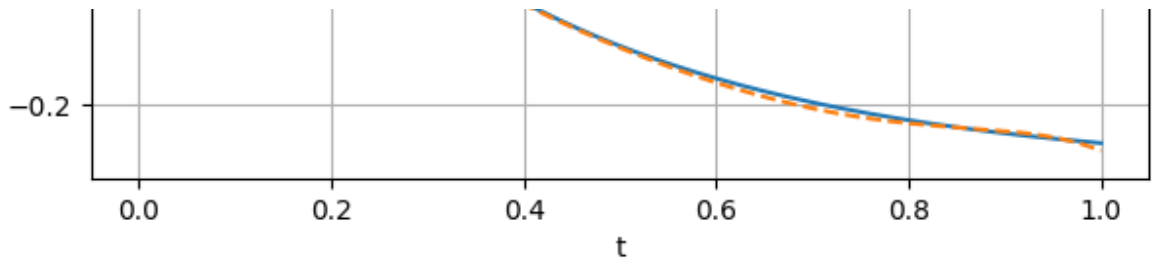
for j in range(3):
    g_hat = evaluate_component_on_grid(add_model, j, t_grid, M=M, ba
    g_hat_c = center_curve_by_train_component(g_hat, comp_means[j])

    g_true = true_functions[j](t_grid)
    g_true_c = g_true - g_true.mean()

    plt.figure(figsize=(7, 4))
    plt.plot(t_grid, g_true_c, label=f"true g_{j+1}, centered")
    plt.plot(t_grid, g_hat_c, "--", label=f"estimated g_{j+1}")
    plt.title(f"Компонента признака x_{j+1}")
    plt.xlabel("t")
    plt.ylabel("g(t)")
    plt.grid(True)
    plt.legend()
    plt.show()
```

Компонента признака x 1





Модель хорошо восстановила компоненты g_1 , g_2 и g_3 .

Оценённые функции почти совпадают с истинными, особенно для x_1 и x_3 .

Небольшие отклонения есть у компоненты x_2 , но общая форма зависимости сохранена.

Итог: аддитивная модель корректно разделила общий сигнал на вклады отдельных признаков.

Генерация неаддитивных данных

Теперь функция содержит взаимодействие признаков:

$$y = \sin(2\pi(x_1 + x_2)) + 0.5x_3 + \text{arepsilon}.$$

Эту зависимость нельзя разложить как сумму отдельных функций только от x_1 , x_2 и x_3 . Поэтому аддитивная модель должна работать хуже, чем более общая модель с взаимодействиями.

```
def generate_nonadditive_data(n_samples=1000, noise_std=0.1, seed=42):
    local_rng = np.random.default_rng(seed)
    X = local_rng.uniform(0, 1, size=(n_samples, 3))
    y_true = np.sin(2 * np.pi * (X[:, 0] + X[:, 1])) + 0.5 * X[:, 2]
    y = y_true + local_rng.normal(0, noise_std, size=n_samples)
    return X, y, y_true
```

Train/test для неаддитивных данных

```
X_na, y_na, y_true_na = generate_nonadditive_data(n_samples=1200, n
X_na_train, X_na_test, y_na_train, y_na_test = train_test_split(
    X_na, y_na, test_size=0.25, random_state=RANDOM_STATE
)

X_na_train_s, X_na_test_s, scaler_na = scale_to_unit_interval(X_na_
```

Обучение моделей на неаддитивной зависимости

Здесь сравниваются три подхода: линейная модель, аддитивная модель и полиномиальная модель с взаимодействиями. Ожидается, что модель с взаимодействиями лучше опишет неаддитивную структуру.

```
lin_na = fit_linear_model(X_na_train_s, y_na_train)
pred_na_lin_train = lin_na.predict(X_na_train_s)
pred_na_lin_test = lin_na.predict(X_na_test_s)

add_na = fit_additive_ridge(X_na_train_s, y_na_train, M=10, alpha=1
pred_na_add_train = predict_additive(add_na, X_na_train_s, M=10, ba
pred_na_add_test = predict_additive(add_na, X_na_test_s, M=10, basi

poly_na, full_na = fit_full_polynomial_model(X_na_train_s, y_na_tra
pred_na_full_train = predict_full_polynomial(poly_na, full_na, X_na
pred_na_full_test = predict_full_polynomial(poly_na, full_na, X_na_
```

Таблица качества на неаддитивных данных

Если полная модель показывает заметно меньшую ошибку, это подтверждает ограничение аддитивного подхода: он хорошо работает для сумм одномерных эффектов, но не всегда способен восстановить взаимодействия признаков.

```

rows = []
for name, pred_train, pred_test, n_features in [
    ("Linear", pred_na_lin_train, pred_na_lin_test, X_na_train.shape[1]),
    ("Additive trigonometric", pred_na_add_train, pred_na_add_test,
     ("Full polynomial degree=5", pred_na_full_train, pred_na_full_test,
    ]:
    tr = regression_metrics(y_na_train, pred_train)
    te = regression_metrics(y_na_test, pred_test)
    rows.append({
        "model": name,
        "n_features": n_features,
        "train_RMSE": tr["RMSE"],
        "test_RMSE": te["RMSE"],
        "train_R2": tr["R2"],
        "test_R2": te["R2"],
    })

pd.DataFrame(rows)

```

	model	n_features	train_RMSE	test_RMSE	train_R2	test_R2
0	Linear	3	0.706549	0.736923	0.083402	0.017764
1	Additive trigonometric	60	0.684345	0.765306	0.140108	-0.059354
	Full polynomial degree=5	15	0.684345	0.765306	0.140108	-0.059354

На неаддитивных данных аддитивная модель работает плохо: test R^2 стал отрицательным.

Линейная модель тоже почти не объясняет зависимость.

Полная полиномиальная модель намного лучше: test $R^2 \approx 0.92$.

когда в данных есть взаимодействия между признаками, аддитивной модели недостаточно. Нужна более общая модель, учитывающая совместное влияние признаков.

Загрузка реальных данных

California Housing — реальный регрессионный датасет. Его удобно использовать для проверки моделей на данных, где истинная функция неизвестна. Если загрузка не работает в среде без интернета, этот блок можно пропустить.

```
from sklearn.datasets import fetch_california_housing

try:
    california = fetch_california_housing()
    X_real = california.data
    y_real = california.target
    feature_names = california.feature_names
    print("Датасет загружен:", X_real.shape)
except Exception as e:
    print("Не удалось загрузить California Housing:", e)
    X_real = None
    y_real = None
    feature_names = None
```

Датасет загружен: (20640, 8)

```
if X_real is not None:
    X_real_train, X_real_test, y_real_train, y_real_test = train_test_split(
        X_real, y_real, test_size=0.25, random_state=RANDOM_STATE
    )

    X_real_train_s, X_real_test_s, real_scaler = scale_to_unit_integers(
        X_real_train, X_real_test, real_scaler
    )

    print("Train:", X_real_train_s.shape)
    print("Features:", feature_names)
```

Train: (15480, 8)
Features: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'Latitude', 'Longitude']

Обучение моделей на реальных данных

На реальных данных нельзя сравнить восстановленные компоненты с истинными, поэтому основное внимание уделяется метрикам на test-выборке и интерпретации графиков компонент.

```
if X_real is not None:
    real_lin = fit_linear_model(X_real_train_s, y_real_train)
    real_lin_train = real_lin.predict(X_real_train_s)
    real_lin_test = real_lin.predict(X_real_test_s)

    real_add = fit_additive_ridge(X_real_train_s, y_real_train, M=8)
    real_add_train = predict_additive(real_add, X_real_train_s, M=8)
    real_add_test = predict_additive(real_add, X_real_test_s, M=8,

    real_poly, real_full = fit_full_polynomial_model(X_real_train_s,
    real_full_train = predict_full_polynomial(real_poly, real_full,
    real_full_test = predict_full_polynomial(real_poly, real_full,
```

Таблица качества на реальных данных

Если аддитивная модель улучшает качество относительно линейной, значит в данных есть нелинейные одномерные эффекты. Если полная модель даёт ещё лучшее качество, это может говорить о наличии взаимодействий между признаками.

```

if X_real is not None:
    rows = []
    for name, pred_train, pred_test, n_features in [
        ("Linear", real_lin_train, real_lin_test, X_real_train_s.sh
        ("Additive ridge polynomial", real_add_train, real_add_test
        ("Full polynomial degree=2", real_full_train, real_full_test
    ]:
        tr = regression_metrics(y_real_train, pred_train)
        te = regression_metrics(y_real_test, pred_test)
        rows.append({
            "model": name,
            "n_features": n_features,
            "train_RMSE": tr["RMSE"],
            "test_RMSE": te["RMSE"],
            "train_R2": tr["R2"],
            "test_R2": te["R2"],
        })
    display(pd.DataFrame(rows))

```

	model	n_features	train_RMSE	test_RMSE	train_R2	test_R2
0	Linear	8	0.721493	0.735615	0.609873	0.591051
1	Additive ridge polynomial	64	0.716359	0.725322	0.615405	0.602415

На реальных данных все модели дают среднее качество.

Линейная модель объясняет около 59% тестовой вариации.

Аддитивная ridge-модель немного лучше линейной: test $R^2 \approx 0.60$.

Полная полиномиальная модель лучше остальных: test $R^2 \approx 0.66$.

Итог: в этих данных есть взаимодействия между признаками, поэтому полная полиномиальная модель работает лучше аддитивной.

Интерпретационные графики для реальных данных

Графики $\hat{g}_j(t)$ показывают, как каждый признак влияет на прогноз при фиксированной аддитивной структуре. Это одно из главных преимуществ аддитивных моделей по сравнению с полностью чёрными ящиками.

```

if X_real is not None:
    t_grid = np.linspace(0, 1, 300)

```

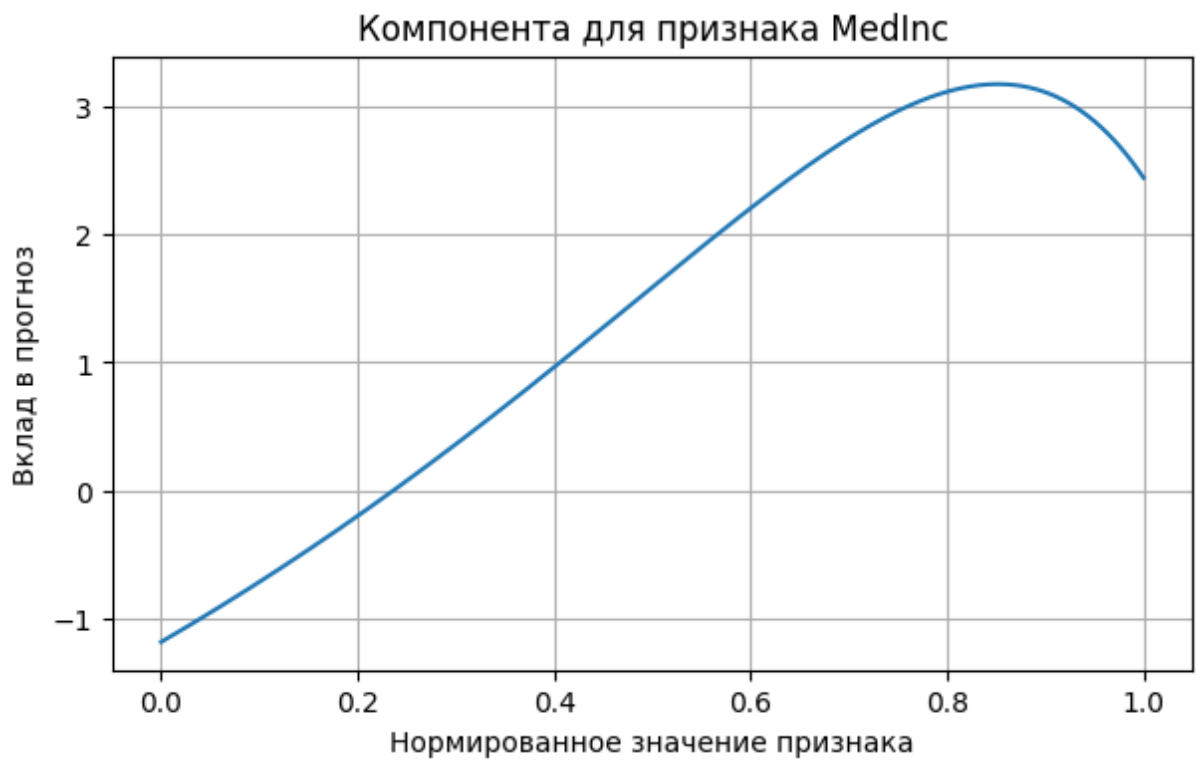
```

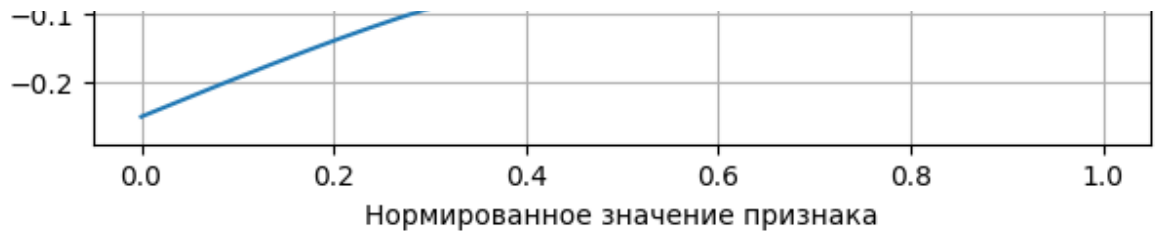
intercept_real_c, _, real_means = centered_additive_components(
    real_add, X_real_train_s, X_real_train_s, M=8, basis_type="
)

for j, name in enumerate(feature_names[:6]):
    g_hat = evaluate_component_on_grid(real_add, j, t_grid, M=8
    g_hat_c = center_curve_by_train_component(g_hat, real_means

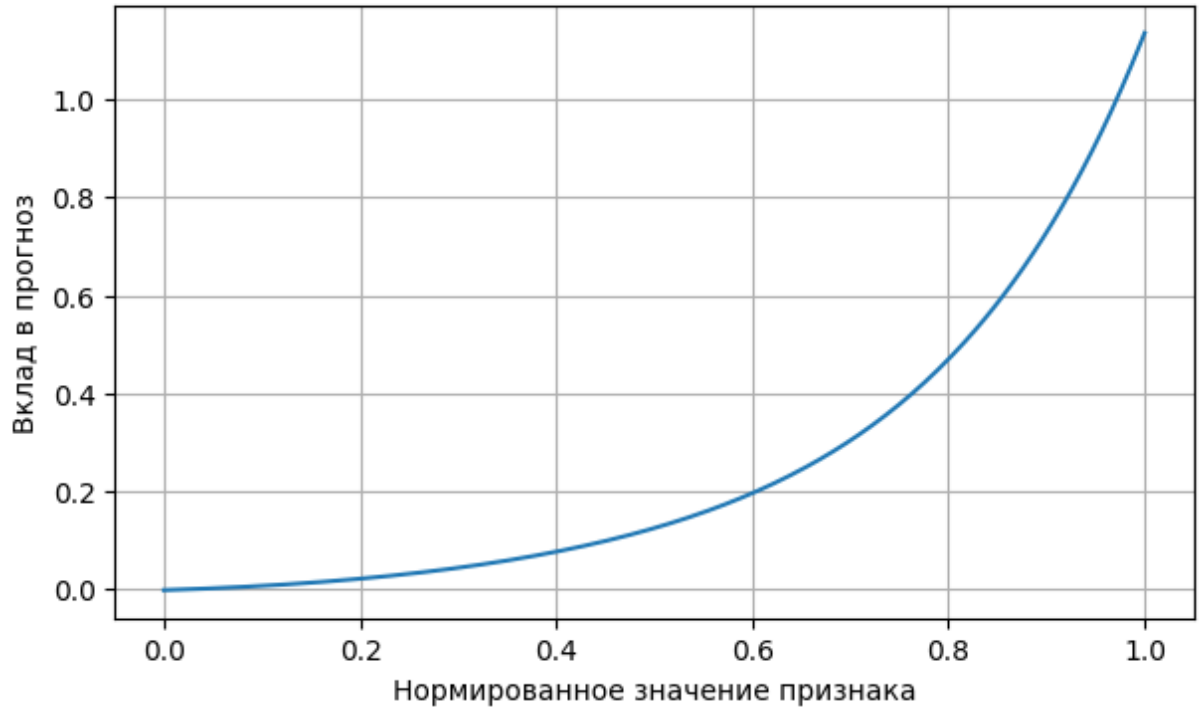
    plt.figure(figsize=(7, 4))
    plt.plot(t_grid, g_hat_c)
    plt.title(f"Компонента для признака {name}")
    plt.xlabel("Нормированное значение признака")
    plt.ylabel("Вклад в прогноз")
    plt.grid(True)
    plt.show()

```

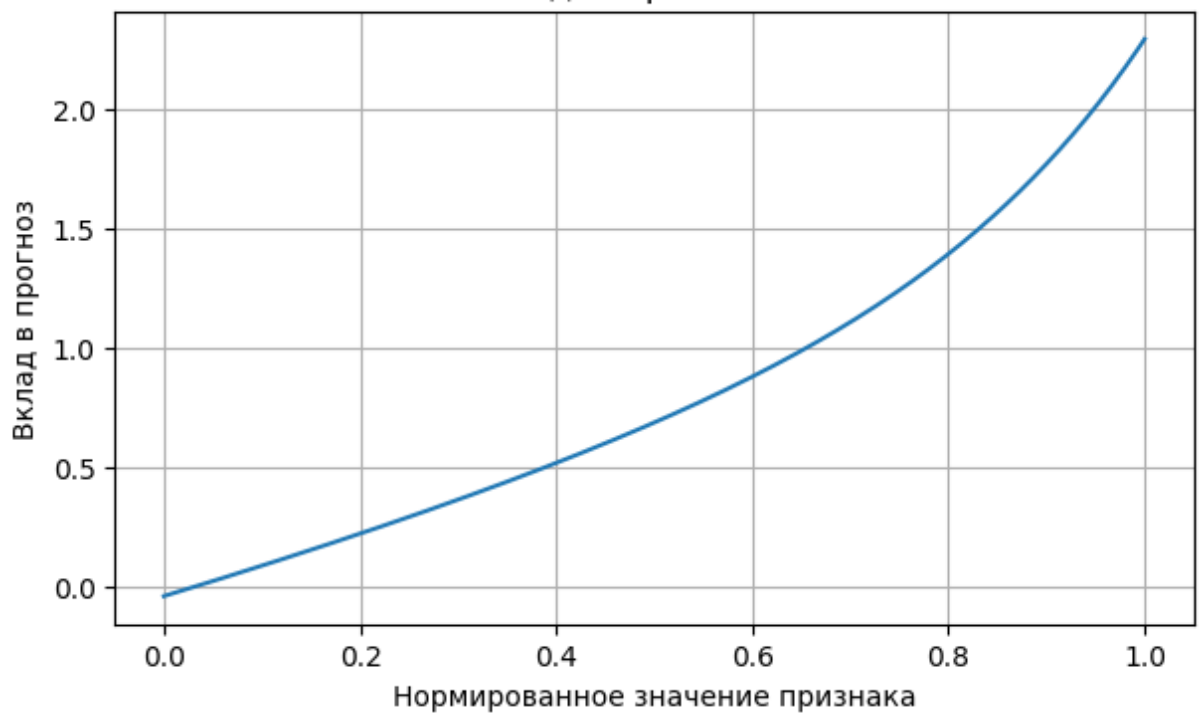




Компонента для признака AveRooms

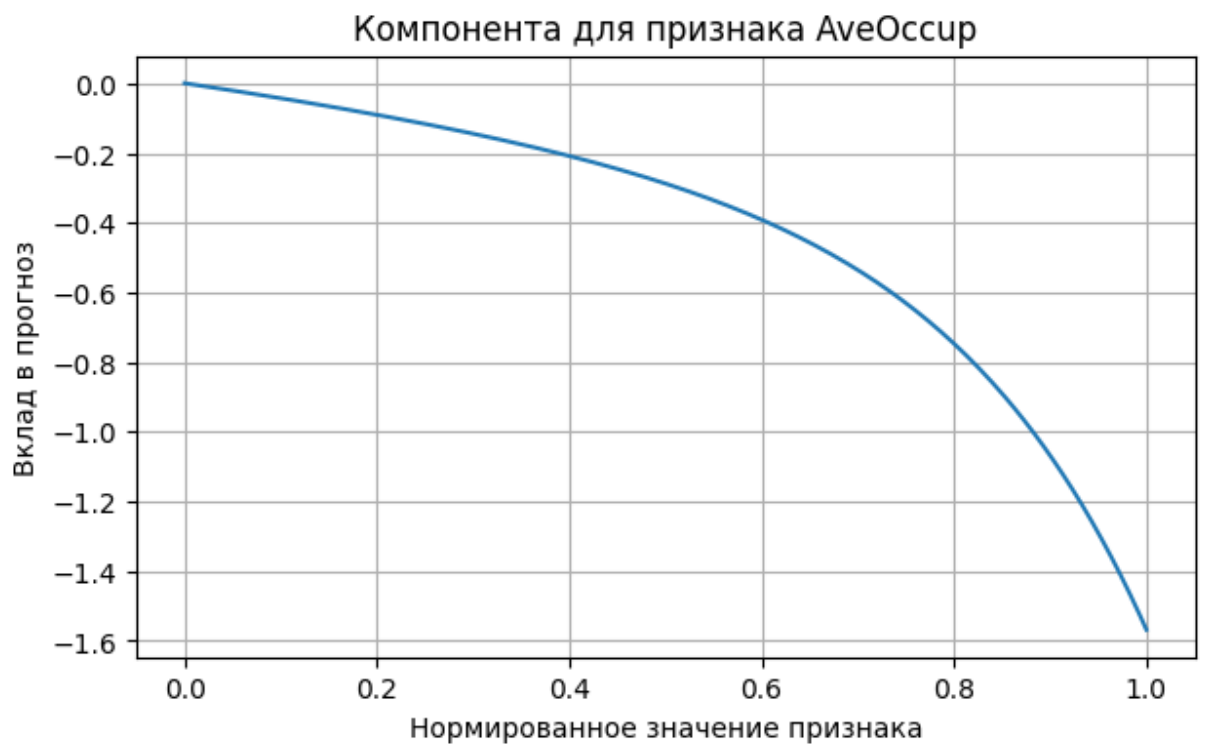
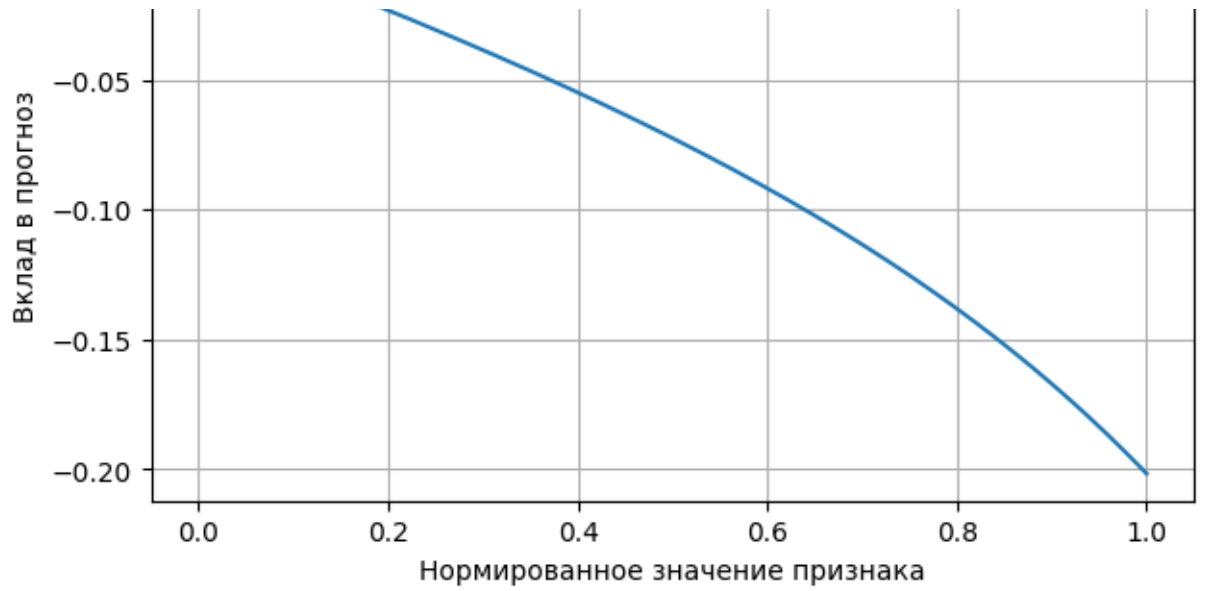


Компонента для признака AveBedrms



Компонента для признака Population





Для синтетических данных модель почти точно восстановила компоненты g_1 , g_2 , g_3 .

Для реальных данных компоненты показывают влияние отдельных признаков: MedInc имеет самый сильный положительный вклад, а HouseAge влияет слабее и заметно растёт только при больших значениях

Вывод по кейсу 7

Аддитивная модель обобщает линейную регрессию: вместо $\beta_j x_j$ используется функция $g_j(x_j)$. После базисного разложения модель линейна по параметрам. МНК подходит для базового варианта, ridge — для регуляризации. Центрирование компонент нужно, чтобы разложение было интерпретируемым. На аддитивных данных модель хорошо восстанавливает отдельные функции. На неаддитивных данных качество ограничено из-за отсутствия взаимодействий. Полная полиномиальная модель может быть точнее, но у неё больше параметров и выше риск переобучения. Разумное число базисных функций выбирается по ошибке на тестовой выборке.

Итоговый вывод по кейсу 1

Ядро $K(x, t) = \min(x, t)$ симметрично, потому что $\min(x, t) = \min(t, x)$. Численная матрица Грама также получается симметричной.

Положительность ядра подтверждается двумя способами: через неотрицательные собственные значения матрицы Грама и через проверку квадратичных форм $z^T G z$. Это согласуется с тем, что данное ядро является ковариацией винеровского процесса.

Аналитический спектр задаётся формулами

$$\lambda_k = \frac{1}{\pi^2(k - 1/2)^2}, \quad \varphi_k(x) = \sqrt{2} \sin((k - 1/2)\pi x).$$

Численная дискретизация через квадратурные веса переводит интегральную задачу в матричную. Сравнение показало, что численные собственные значения и функции хорошо совпадают с аналитическими. При увеличении числа узлов сетки ошибка уменьшается, что подтверждает сходимость метода.

Усечённое разложение ядра

$$K_m(x, t) = \sum_{k=1}^m \lambda_k \varphi_k(x) \varphi_k(t)$$

становится точнее при росте m . Главный вклад дают первые собственные пары, потому что собственные значения быстро убывают.

Итоговый вывод по кейсу 7

Аддитивная модель является естественным обобщением линейной регрессии. В линейной модели вклад признака имеет вид $\beta_j x_j$, а в аддитивной модели он заменяется на произвольную одномерную функцию $g_j(x_j)$.

После разложения компонент по базису задача становится линейной по параметрам, поэтому её можно решать методом наименьших квадратов:

$$\sum_i (y_i - \hat{y}_i)^2 \rightarrow \min.$$

Для борьбы с переобучением используется ridge-регрессия со штрафом

$$\alpha \sum_r \theta_r^2.$$

На аддитивных синтетических данных такая модель должна выигрывать у линейной регрессии, потому что умеет восстанавливать нелинейные одномерные эффекты. При этом она проще и понятнее полной полиномиальной модели с взаимодействиями.

Центрирование компонент делает разложение однозначным:

$$\sum_i \hat{g}_j(x_{ij}) = 0.$$

После этого графики $\hat{g}_j(t)$ можно интерпретировать как отдельный вклад каждого признака.

На неаддитивных данных аддитивная модель ограничена, потому что не содержит взаимодействий между признаками. Поэтому для функций вида $\sin(2\pi(x_1 + x_2))$ или $x_1 x_2 + x_3$ более общая модель с взаимодействиями обычно должна показывать лучшее качество.

